Revisiting ContextAgent: A Reproducibility Study of Context-Aware Proactive LLM Agents under Resource Constraints

Anonymous authors
Paper under double-blind review

Abstract

Proactive Large Language Model (LLM) agents decide on their own when to offer assistance, which requires them to evaluate context continuously and makes their inference cost a first-order concern. We reproduce *ContextAgent* (Yang et al., 2025b), a context-aware proactive agent evaluated on *ContextAgentBench*, using the authors' released code and data split, and then examine whether its results hold under common efficiency techniques: parameter-efficient fine-tuning choices, base-model quantisation and pruning. The central claim of the original work—that supervised fine-tuning with persona context and thought traces clearly outperforms in-context learning baselines—is confirmed for all three base models, and the in-context baselines reproduce closely. However, the fine-tuned models score substantially *higher* than reported, and the gap depends on a discrepancy we identified between the paper (learning rate 10^{-4}) and the released training configuration (5×10^{-5}): with the paper's stated learning rate, tool-calling F1 rises from the reported 0.626/0.647/0.645 to 0.777/0.685/0.740 (mean of three seeds) for Llama-3.1-8B, DeepSeek-R1-7B and Qwen2.5-7B. Training with the effective batch size of eight-GPU data-parallel training reproduces the reported Llama score, but not those of DeepSeek and Qwen. Beyond reproduction, we find that 4-bit NF4 quantisation of the base model roughly halves inference memory without a statistically significant change in tool-calling F1 for any of the three models (differences of at most 3 points), at the cost of up to 53% higher latency with the bitsandbytes runtime. When the pruning mask is preserved, unstructured Wanda pruning maintains task performance up to 30% sparsity; at 50% and 2:4 sparsity, restricting the recovery update to the unpruned weights reduces the loss in tool-calling F1 from 0.10 and 0.27 to 0.03 and 0.09, and with the paper's learning rate removes it at 50% unstructured and 4:8 sparsity, but without sparse kernels pruning saves neither memory nor latency. Structured pruning of feed-forward layers reduces both memory and latency, but degrades performance already at 10% sparsity, whereas directly fine-tuning a 1.5B model (Qwen2.5-1.5B) at the full sequence length matches the released Qwen2.5-7B score (0.707 F1, three seeds) at a quarter of the inference memory (3.8 GB) and about half the latency. Overall, while the qualitative conclusions of *ContextAgent* hold, the absolute performance of the fine-tuned reasoner is highly sensitive to under-documented training hyperparameters, and post-pruning recovery must preserve the pruning mask, because merging an unconstrained adapter restores dense weights. We release code, configurations, model predictions and score files to reproduce all results.

1 Introduction

Most LLM-based agents are reactive: they produce an answer or perform an action only after receiving a prompt from the user. A different line of work studies proactive agents, which use available context to decide whether assistance is needed and may act without being explicitly instructed to do so (Lu et al., 2024). This changes the role of the agent from answering requests to taking the initiative, which also affects how users perceive and work with the system (Diebel et al., 2024). It also changes the practical cost of running

the system. A reactive agent performs inference only on user interaction, whereas a proactive agent runs continuously or at regular intervals to monitor context, decide whether intervention is needed and possibly call external tools. A proactive agent may therefore perform well on a benchmark while still being too costly to operate in practice.

ContextAgent (Yang et al., 2025b) is a recent framework for context-aware proactive LLM agents that combines sensory context, persona information, proactive prediction and tool calling. It is accompanied by *ContextAgentBench*, a benchmark with 1,000 samples across nine daily scenarios and 20 tools, and by a public implementation. The original work focuses on capability: it shows that supervised fine-tuning of 7–8B models with persona context and thought traces outperforms in-context learning baselines on proactive prediction and tool calling. The cost of maintaining this behaviour is not evaluated.

This paper is a reproducibility study of *ContextAgent*. We first reproduce the main results on *ContextAgentBench* with the authors' code and data split. We then test whether the conclusions hold, and at what cost, when the fine-tuned reasoner is made cheaper to train and run with widely used efficiency techniques: Low-Rank Adaptation (LoRA) configuration choices, base-model quantisation (LLM.int8 and 4-bit NormalFloat (NF4)) and pruning (unstructured, semi-structured and structured). Our contributions are:

- **Reproduction of the main claims.** The in-context learning baselines reproduce within a few points of the reported values, and the supervised *ContextAgent* configuration outperforms them for all three base models, confirming the central claim (Section 6.1).
- **A paper–code discrepancy and its effect.** The paper reports a learning rate of 10^{-4}, whereas the released training configuration uses 5×10^{-5}. Both settings yield higher fine-tuned scores than reported, and the paper's setting yields the highest; for example, Llama-3.1-8B reaches a tool-calling F1 of 0.777 (mean of three seeds) against the reported 0.626 (Table 2).
- **Robustness under efficiency constraints.** We quantify the cost–performance trade-off of LoRA rank and target modules, quantisation and pruning for the *ContextAgent* reasoner, measuring training and inference memory, latency and throughput alongside the original task metrics (Section 6.2).

The study is limited to *ContextAgent* as the framework and *ContextAgentBench* as the evaluation setting, and it covers the context-aware reasoner only; the perception stage (vision-language model and speech recognition) is outside the cost measurements.

2 Related Work

2.1 Proactive LLM Agents

Most LLM agents are reactive systems, where the model acts after receiving an explicit user prompt. ReAct (Yao et al., 2023) combines reasoning and action in an iterative process, which is relevant for tool use and multi-step decision making, but the agent is still given a task or observation to respond to. A proactive agent must additionally decide whether assistance should be provided at all.

Lu et al. (2024) frame proactivity as a prediction problem in which the model learns when proactive suggestions are likely to be accepted by users. Yang et al. (2025a) study proactivity in a deployed setting, where sensory context is gathered through on-demand perception and used together with LLM reasoning to provide proactive assistance on AR glasses. *ContextAgent* (Yang et al., 2025b) combines sensory context, persona information, proactive prediction and tool calling, and evaluates both whether the agent should act and how it should act. These systems primarily target proactive capability; their cost of operation is not the main subject of evaluation.

2.2 Efficient Model Adaptation and Compression

Parameter-efficient fine-tuning methods such as LoRA reduce training cost by freezing the base model and training a small set of adapter parameters (Hu et al., 2021). Quantised Low-Rank Adaptation (QLoRA)

extends this approach by quantising the frozen model during training (Dettmers et al., 2023), and LLM.int8 provides 8-bit inference with outlier handling (Dettmers et al., 2022). Pruning removes parameters or structures from an existing model (Cheng et al., 2024; Sun et al., 2024; Fang et al., 2023), while Knowledge Distillation (KD) trains a smaller student model to imitate a larger teacher (Hinton et al., 2015). Surveys report these as the most common approaches for reducing the cost of LLMs (Kim et al., 2025; Dantas et al., 2025), but they are usually evaluated on reactive tasks. Whether they preserve the behaviour a proactive agent needs, namely calibrated proactive scores, a fixed structured output format and valid tool arguments, is the question this paper addresses for *ContextAgent*.

2.3 Reproducibility Studies

Independent reproduction of published machine learning results, as promoted by the ML Reproducibility Challenge and the reproducibility tracks of information retrieval venues, regularly reveals undocumented choices and discrepancies between papers and released code. Following the ACM terminology, our study is a *reproduction* (different team, same artefacts) of the *ContextAgentBench* results, extended with a *generalisation* to resource-constrained settings.

3 Background

This section introduces the proactive agent setting, the *ContextAgent* framework and the *ContextAgentBench* benchmark.

3.1 Proactive LLM Agents

A proactive LLM agent differs from a reactive agent in the decision problem it solves. A reactive agent receives an explicit instruction from the user and produces an output. A proactive agent instead uses available context to decide whether assistance is needed and may act without being directly prompted (Lu et al., 2024). A simple way to view such a system is as operating on a sequence of contextual states $c_1, c_2, \ldots, c_t$, where each c_t may include observations from the environment, user-related information, and the current system state.

At each configurable sampling step t, the agent computes $f(c_t) = (p_t, a_t)$, where p_t represents the estimated need for assistance and a_t is the planned action. If $p_t < \theta$, where θ is an adjustable threshold, the agent stays passive. If $p_t \geq \theta$ the agent begins assistance. In practice a_t may be a direct response, a tool call or a sequence of tool calls followed by a response.

This also helps explain why proactive agents have a different cost profile than reactive agents. A reactive system performs inference only when prompted, while a proactive system may need to evaluate context repeatedly even when the user is not requesting it. The total cost is therefore shaped not only by the complexity of a single inference step, but also by how often the decision process is triggered and by how much computation is needed to turn these observations into usable context.

3.2 ContextAgent

This general idea is made concrete in *ContextAgent* (Yang et al., 2025b), which combines sensory perceptions and persona context in order to predict the need for proactive assistance and call external tools when needed. The agent task is formalised as:

$$(T, PS, TC, R) = A(S, P) \tag{1}$$

where A is the agent, S denotes sensory perceptions and P denotes persona context. The outputs are thought traces T, a proactive score PS, a planned tool chain TC and a final response R. Proactive assistance is triggered only when $PS \geq \theta$.

This maps directly to the general framework $f(c_t) = (p_t, a_t)$ from Section 3.1, where $c_t = (S, P)$, $p_t = PS$, and $a_t = (TC, R)$.

The framework has two main parts: proactive-oriented context extraction and context-aware proactive reasoning. The first part is tasked with turning raw sensory input into a representation that is easier for the LLM to use. The second part then uses this representation together with persona context to predict whether assistance is needed and which tools should be used.

The sensory input S is viewed as multimodal and is described as including egocentric video, audio and smartphone notifications. From this input, the framework constructs a context representation $C = (C_V, C_A, N)$, where C_V is visual context extracted from video, C_A is audio context extracted from speech, and N denotes textual information from smartphone notifications that is incorporated directly into the final context representation.

For video, *ContextAgent* uses a vision language model with in-context learning to generate visual context C_V. For audio, it uses speech recognition to generate audio context C_A. Persona context P is also used, since the same sensory situation can imply different needs depending on a user's preferences, identity and past behaviour.

After extracting context, the framework applies a context-aware reasoner:

$$(T, PS, TC) = A_S(C, P) \tag{2}$$

where A_S is an LLM that takes sensory context C and persona context P as input and produces thought traces, a proactive score and a planned tool chain. The planned tool chain is a sequence of tool calls with arguments. A simple way to represent this is:

$$TC = \big((t_i, a_i)\big)_{i=1}^{m} \qquad t_i \in \mathcal{T} \tag{3}$$

where $\mathcal{T}$ is the set of available tools, t_i is the selected tool and a_i is the corresponding argument set.

The reasoning model is trained with supervised fine-tuning on a dataset $\mathcal{D}_{\text{SFT}} = \{(X_i, T_i, Y_i)\}_{i=1}^{n}$ where X_i contains the sensory context C_i and persona context P_i, T_i is the thought trace, and Y_i contains the proactive score PS_i and planned tool chain TC_i.

During inference, if the predicted score satisfies $PS \geq \theta$, the planned tools are then executed in sequence. The outputs are then combined with the sensory context, persona context and thought traces to generate the final response.

3.3 ContextAgentBench

ContextAgentBench is the benchmark used in the *ContextAgent* paper to evaluate context-aware proactive agents. Each sample contains seven parts:

$$(S, C, P, T, PS, TC, R) \tag{4}$$

where S is the sensory perception, C is the contextual summary, P is the persona context, T is the thought trace, PS is the proactive score, TC is the planned tool chain, and R is the final response. This means that the benchmark does not only ask if an agent should act. It also includes the context used for the decision, the reasoning trace, the planned use of tools and the final response.

The benchmark is thus a dataset

$$\mathcal{B} = \{(S_i, C_i, P_i, T_i, PS_i, TC_i, R_i)\}_{i=1}^{N} \tag{5}$$

with $N = 1000$. The original paper also introduces *ContextAgentBench-Lite*, with 300 human-verified samples paired with raw sensor data such as egocentric video and audio; we do not use it. The proactive score is defined on a five-point scale, with $PS_i \in \{1, 2, 3, 4, 5\}$, where 1 means that no proactivity is required and 5 means a high level of proactivity.

For non-proactive cases, where $PS_i \in \{1, 2\}$, both the planned tool chain and final response are set to $TC_i = \texttt{None}$ and $R_i = \texttt{None}$. For the proactive cases, the tool chain may contain several tool calls. If $\mathcal{T}$ denotes the set of tool types, then it can be written as:

$$TC_i \in \mathcal{T}^{\leq 5} \cup \{\texttt{None}\} \tag{6}$$

since the benchmark contains 20 tool types and each sample may involve up to five tools.

The benchmark is built in two stages. First, the paper creates 200 human-written seed examples from proactive and non-proactive situations in daily life. These samples include contextual summaries, persona information when needed, proactive scores, thought traces and planned tool chains for proactive cases. Each sample is cross reviewed for plausibility and format. Second, the benchmark is expanded through an automated diversification that generates additional samples with LLMs. Generation is conditioned either on the scenario or on the proactive score. The resulting samples are checked by annotators for rationality and a script to verify the data format and tool arguments.

The released benchmark comes with a fixed split of 600 training and 400 test samples, which we use throughout. Because it provides labels for proactive scores, tool chains and arguments, the original task setting can be reproduced before introducing measures of computational cost.

4 Methods

This section describes the reproduction protocol and the efficiency techniques applied to the *ContextAgent* reasoner.

4.1 Reproduction Protocol

We reproduce the main *ContextAgentBench* experiment of Yang et al. (2025b) (their Table 1) for the three base models reported there: `Llama-3.1-8B-Instruct`, `Qwen2.5-7B-Instruct` and `DeepSeek-R1-7B`. We use the released code, prompts, data split and evaluation scripts; deviations are listed in Section 5.3. We test the following claims:

- **C1** Supervised fine-tuning with thought traces and personas (*ContextAgent*) outperforms all in-context learning baselines on both proactive prediction and tool calling.
- **C2** Adding persona context and thought traces to in-context learning (ICL-All) improves over Vanilla ICL.
- **C3** The reported absolute scores can be obtained with the released artefacts.

All models are evaluated with the original metrics and evaluation code: Proactive Prediction Accuracy (Acc-P), Missed Detection rate (MD), False Detection rate (FD) and Root Mean Square Error (RMSE) for proactive prediction, and precision, recall, F1-score and Argument Accuracy (Acc-Args) for tool calling (Section 5.6). For the configurations behind the claims (the released learning rate 5×10^{-5} and the paper's 10^{-4}), we train three seeds each and compare configurations with a paired bootstrap over the 400 test samples (Section 5.6); to explain deviations from the reported scores, we additionally test the effective batch size of eight-GPU data-parallel training (Section 5.3).

4.2 Selected Efficiency Techniques

We select techniques that are widely used to make LLMs cheaper to train and run. Surveys identify quantisation, pruning, knowledge distillation and parameter-efficient fine-tuning as the most common approaches for reducing computational cost and memory usage while maintaining model performance (Kim et al., 2025; Dantas et al., 2025). These techniques are well established in reactive settings; we assess whether their benefits carry over to the *ContextAgent* reasoner, which must produce calibrated proactive scores, a fixed structured output format and valid tool arguments. Knowledge distillation into 1–1.5B student models was also attempted; because it failed for a technical reason (sequence truncation), we instead fine-tune the same small models directly at the full sequence length and report them as an additional efficiency reference in Section 7.2.

4.3 Low-Rank Adaptation (LoRA)

ContextAgent already fine-tunes its reasoner with LoRA (rank 8, all linear layers). We therefore do not compare LoRA against full fine-tuning, but study how the LoRA configuration (rank, target modules, learning rate) and base-model quantisation affect task performance and cost.

LoRA freezes the pre-trained weights and injects trainable rank-decomposition matrices into each targeted layer of the *Transformer* (Hu et al., 2021). For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update is constrained to a low-rank decomposition $W_0+\Delta W = W_0+BA$, where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ and $r \ll \min(d,k)$. The forward pass computes

$$h = W_0x + \Delta Wx = W_0x + BAx. \tag{7}$$

This reduces the number of trainable parameters and therefore the optimiser-state memory, while performing comparably to full fine-tuning on many tasks (Hu et al., 2021). The rank r and the set of targeted weight matrices are the main capacity choices.

4.3.1 Quantisation and QLoRA

QLoRA trains LoRA adapters on top of a quantised, frozen base model (Dettmers et al., 2023). Base-model quantisation compresses the frozen weights into lower-precision data types, reducing the memory required to load the model during both training and inference. Since only the adapters are trained, aggressive quantisation of the base weights does not directly affect their optimisation. We use two quantisation levels, each with 16-bit adapters: 8-bit LLM.int8 and 4-bit NF4.

4-bit NormalFloat (NF4). NF4 is a 4-bit data type designed for zero-mean, normally distributed weights: it maps weights to 4-bit bins with equal expected probability mass, minimising quantisation error under that distributional assumption (Dettmers et al., 2023). This allows 7–8B models to be fine-tuned under tight memory limits while maintaining performance close to 16-bit baselines.

8-bit LLM.int8. LLM.int8 stores weights in 8-bit precision and preserves task performance by isolating outlier features, which are processed in 16-bit precision, while the vast majority of matrix multiplications are performed in 8-bit (Dettmers et al., 2022). Compared to NF4, it trades smaller memory savings for higher numerical fidelity.

4.4 Pruning

Pruning removes parameters or computational structures that contribute little to predictive performance. Unlike LoRA, which adds low-rank updates alongside the frozen weights, pruning directly modifies the original network. Let $\boldsymbol{\theta}$ denote the parameters of a model and $\mathbf{m} \in \{0,1\}^{|\boldsymbol{\theta}|}$ a binary pruning mask. The pruned model is $f(x; \mathbf{m} \odot \boldsymbol{\theta})$, and the objective is to minimise performance degradation under a given sparsity constraint (Cheng et al., 2024). For LLMs, pruning can reduce memory and computation at inference time, which could enable proactive agents on constrained hardware. Pruning strategies are commonly divided into unstructured pruning, which removes individual parameters, and structured pruning, which removes entire parameter groups such as channels, neurons or attention heads.

4.4.1 Unstructured Pruning

Pruning performance depends on how well the importance of weights is estimated. Importance scores are computed using a calibration dataset that reflects the inputs seen during deployment. This data is used to collect statistics such as activation magnitudes, allowing data-aware pruning without full retraining. We take the calibration data from the *ContextAgentBench* training split so that it reflects the task distribution.

Unstructured pruning removes weights independently without enforcing any pattern, allowing the model to retain the most important parameters at a given sparsity level. As a result, it typically retains more accuracy than more constrained pruning methods at the same sparsity, but it does not reduce memory or latency unless sparse kernels are used.

Among data-aware methods, Pruning by **W**eights **and A**ctivations (WANDA) (Sun et al., 2024) is an effective one-shot pruning approach. It estimates the importance of each weight by combining its value with the activation statistics of its input feature.

Consider a linear layer with weight matrix $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$. Given a calibration dataset, the input activations are collected into a matrix $\mathbf{X} \in \mathbb{R}^{(N \times L) \times C_{\text{in}}}$, where N is the batch size and L is the sequence length. The importance score for each weight w_{ij} is defined as:

$$S_{ij} = |w_{ij}| \cdot \|\mathbf{X}_j\|_2, \tag{8}$$

where $\mathbf{X}_j$ is the j-th column of $\mathbf{X}$, representing the activation of the j-th input feature.

Weights with the lowest importance scores are removed. This approach extends magnitude-based pruning by also considering how often a feature is used. By aggregating activations across the calibration data, the method captures usage patterns that are important in real tasks.

To provide a baseline, WANDA is compared with standard magnitude pruning, which uses only the size of weights. WANDA typically performs better at higher sparsity levels, where magnitude pruning may remove weights that are still important during inference.

These importance scores are extended to the semi-structured setting by enforcing N:M sparsity constraints. In this setup, each weight matrix is divided into blocks of M parameters, and only the N most important weights are kept in each block.

This semi-structured design balances accuracy and hardware efficiency. Results for the 2:4 and 4:8 sparsity patterns are reported in Appendix C.

4.4.2 Structured Pruning

Torch-Pruning is employed for structured pruning (Fang et al., 2023), a DepGraph-based framework that removes structurally coupled parameter groups while preserving model consistency. Pruning is applied post-training to a pretrained model, targeting primarily the feed-forward Multilayer Perceptron (MLP) layers, while attention head pruning is disabled to preserve architectural stability.

Unlike unstructured pruning, structured pruning operates on groups of parameters that are interdependent across layers. The DepGraph framework explicitly models these dependencies and ensures that coupled parameters are pruned jointly, enabling consistent removal of neurons or hidden dimensions without breaking the computational graph. As a result, pruning is performed at the group level rather than on individual parameters.

To estimate group importance, two criteria are evaluated within the same pruning framework. The first, `GroupMagnitudeImportance`, ranks parameter groups by the aggregated L2 norm of their weights, where the L2 norm captures the group's overall magnitude. The second, `ActivationImportance`, uses activation statistics collected from a calibration dataset through forward passes, reflecting how frequently and strongly each group contributes to the model's outputs.

Equal pruning ratios are applied across MLP layers, while attention heads remain unpruned, allowing for the isolation of the impact of structured sparsity in feed-forward components. This choice is made based on preliminary experiments in which attention head pruning revealed severe degradation in model performance, even at low sparsity levels. Consequently, head pruning is disabled in the final configuration, and structured pruning is applied exclusively to feed-forward MLP layers.

4.4.3 Pruning Shortcomings and Recovery Phase

Pruning modifies the model's optimisation by removing parameters or structural components that contribute to learned representations. As a result, even importance-aware pruning strategies often introduce accuracy degradation, distribution shifts in intermediate activations, and reduced calibration, particularly at higher sparsity levels. These effects are more pronounced in structured pruning, where removing entire heads or hidden dimensions disrupts inter-layer dependencies. To mitigate these issues, a recovery phase is commonly

applied after pruning (Cheng et al., 2024), consisting of Supervised Fine-Tuning (SFT) on task-specific data. This recovery step allows the model to adapt to the induced sparsity through LoRA-based fine-tuning, partially restoring performance while preserving the efficiency gains introduced by pruning. In our experiments, all pruned models underwent the same recovery procedure to ensure a fair comparison across pruning strategies and sparsity regimes.

5 Experimental Setup

5.1 Data and Configurations

All experiments use the released *ContextAgentBench* split of 600 training samples (`cab_train_sft`) and 400 test samples (`cab_test`) (Yang et al., 2025b). Each sample contains the following fields:

- *Context information:* visual, audio and smartphone-notification context extracted from wearable sensors.
- *Personas:* a description of the user in the specific scenario.
- *Thoughts:* the reference thought trace.
- *Proactive score:* the amount of proactivity needed, from 1 (none) to 5 (high).
- *Tools:* the tool chain, including arguments, that the agent should call.
- *Response:* the agent's final free-text response.

The combinations of these fields define the methods in Table 1, following the original paper. *Vanilla ICL* uses the context information only; *CoT* adds thought traces; *ICL-P* adds personas; and *ICL-All* adds both. *ContextAgent* is the supervised configuration: the model is fine-tuned with LoRA adapters on all linear layers using sensory context, thought traces and personas, which makes it the fine-tuned counterpart of *ICL-All*.

5.2 Implementation Details

All fine-tuning uses the LLaMA-Factory configuration released with *ContextAgent*, with a sequence cut-off of 2048 tokens, 5 epochs, a cosine learning-rate schedule with 10% warm-up, bfloat16 mixed precision, gradient checkpointing and an effective batch size of 8 (per-device batch size 1, 8 gradient-accumulation steps). The default LoRA configuration is rank $r = 8$, $\alpha = 2r$, all linear layers, and learning rate 5×10^{-5}, as in the released code. Experiments ran on a single NVIDIA L4 GPU with 24 GB of memory. The software environment used PyTorch 2.7.0, Transformers 4.52.1, PEFT 0.15.2, LLaMA-Factory 0.9.3.dev0, and bitsandbytes 0.45.1. Quantised runs used the bitsandbytes backend for 4-bit quantisation. Generation used top-p/top-k sampling with $p = 0.7$ and $k = 50$, temperature 0.95, one beam, and a maximum of 1024 new tokens. Following the original evaluation protocol of Yang et al. (2025b), which also evaluates each configuration from a single run on the fixed test split without reporting repeated seeds or variance, we evaluate most configurations from a single completed training run ($k = 1$). For the two configurations behind the reproduction claims (LoRA $r = 8$ with the released learning rate 5×10^{-5} and the paper's 10^{-4}), we additionally train seeds 1 and 2 and report the mean and standard deviation over three runs; the seed-to-seed standard deviation of tool-calling F1 is 0.005–0.015, smaller than the evaluation uncertainty of a single run. With 400 test samples, a single evaluation has a 95% bootstrap confidence interval of about ± 0.035 for the tool-calling F1-score and about ± 0.05 for Acc-Args; we test differences between configurations explicitly (Section 5.6) rather than interpreting differences of a few hundredths.

5.3 Deviations from the Original Setup

- **Learning rate discrepancy.** The ContextAgent paper reports an SFT learning rate of 10^{-4} (Section 5.1), whereas the released LLaMA-Factory training configuration files (`llama3_lora_sft.yaml`, `qwen_lora_sft.yaml`) specify 5×10^{-5}. Rather than assuming an error, we treat this as

a paper–code discrepancy and evaluate both settings: we adopt the released configuration (5×10^{-5}) for our baseline reproduction and evaluate the paper's stated value (10^{-4}) in a separate learning-rate experiment (Table 2, Table 7). With the paper's value, all three models exceed the reported scores; with the released value, Llama and Qwen exceed them while DeepSeek falls below (0.596 versus 0.647). 10^{-4} improves over 5×10^{-5} for all three models.

- **Hardware and batch size.** The paper reports training on eight A6000 GPUs. The released configuration uses per-device batch size 1 with 8 gradient-accumulation steps; if it was run data-parallel on eight GPUs, the effective batch size was 64 rather than 8, corresponding to eight times fewer optimiser steps. We train on a single GPU with an effective batch size of 8. To test whether this explains the difference to the reported scores, we additionally train each model with an effective batch size of 64 (per-device batch size 1, 64 accumulation steps), which reproduces the optimiser-step count of eight-GPU data-parallel training (Table 3).
- **Fine-tuning details.** Our LoRA and learning-rate experiments add a LoRA dropout of 0.05 and gradient checkpointing, which are not set in the released configuration; the LoRA scaling ($\alpha = 2r$) and all other hyperparameters follow the released files. For DeepSeek-R1-7B we use LLaMA-Factory's `deepseek3` chat template instead of the released `deepseek` template. The quantised (QLoRA) runs use per-device batch size 2 with 4 accumulation steps; the effective batch size of 8 is unchanged. To check that these settings do not affect the results, we also trained each model twice with the exact released configuration. For DeepSeek, these runs are equivalent to ours within ± 0.03 on all metrics (F1-score difference -0.004, 95% CI $[-0.030, 0.023]$). For Llama, the released configuration scores slightly lower (-0.021, $[-0.042, 0.000]$), a difference at the edge of significance and far smaller than the effects discussed below. For Qwen, the two runs are equivalent within ± 0.03 in F1-score (difference 0.000, 95% CI $[-0.021, 0.021]$).
- **Random seeds.** The released configuration sets no seed, so LLaMA-Factory's default (42) applies. Our original runs used this default. For the two main learning rates (5×10^{-5} and 10^{-4}) we additionally train seeds 1 and 2 and report the mean and standard deviation over the three runs.
- **Configuration search.** The released split has no validation set. The LoRA sweeps in Section 6.2 are therefore evaluated on the test split and should be read as an oracle sensitivity analysis; the "best" configurations are optimistic estimates and are not used to support the reproduction claims, which rely only on the released and paper configurations.

5.4 Quantisation Settings

Quantised runs load the base model with bitsandbytes before injecting 16-bit adapters. The 8-bit runs use LLM.int8 (Dettmers et al., 2022); the 4-bit runs use NF4 with double quantisation (Dettmers et al., 2023). The trainable adapters are kept in bfloat16. All other parameters are identical to the corresponding 16-bit run.

5.5 Pruning Settings

Pruning is evaluated on `Llama-3.1-8B-Instruct`, the strongest model in our reproduction, at sparsity ratios of 10%, 30% and 50% for unstructured and structured pruning, and with 2:4 and 4:8 patterns for semi-structured pruning (Appendix C). WANDA and magnitude pruning are compared for unstructured and semi-structured sparsity, while structured pruning uses group magnitude and activation-based importance. Data-aware criteria use calibration samples from the training split with a fixed sequence length of 2048 tokens: 256 samples for WANDA and 512 for activation-based structured pruning (the defaults of the respective implementations).

Evaluations immediately after pruning showed severe degradation at moderate and high sparsity, so all pruned models undergo the same LoRA-based recovery fine-tuning before evaluation, using the default configuration for unstructured and semi-structured pruning and a learning rate of 2×10^{-5} for structured pruning. Merging the recovery adapters into the pruned weights (`merge_and_unload()`) adds the dense low-rank update BA

to every adapted matrix and thereby fills the pruned entries: after merging, the WANDA models contain no zero weights (0.0% sparsity, against 10–50% before merging). We therefore record the pruning mask before merging and re-apply it afterwards, so that the evaluated models have the stated sparsity; Table 12 reports results with the re-applied mask; results without it are not reported, because those models are dense. Structured pruning removes entire channels, so its sparsity is preserved by merging. Pruned weights are stored in dense format, so unstructured and semi-structured sparsity is not exploited by the inference kernels. Because re-applying the mask discards the part of the recovery update that falls on pruned weights, we additionally run a *mask-aware* recovery for the 50% and semi-structured settings: the LoRA update is restricted to the weights that survived pruning, $\Delta W = (BA) \cdot s \odot M$ with $M = \mathbb{1}[W \neq 0]$, by replacing the forward pass of the LoRA layers. All other settings are identical to the original recovery, and merging the adapter then preserves the sparsity exactly.

5.6 Metrics

All configurations are evaluated on the 400 test samples with the original evaluation code.

Proactive prediction. Acc-P measures whether the predicted need for proactive assistance (proactive score above the threshold) matches the ground truth. The MD is the fraction of samples where assistance was needed but not predicted, and the FD is the fraction where assistance was predicted but not needed. RMSE measures the error of the predicted proactive score on the 1–5 scale.

Tool calling. Precision, recall and F1-score compare predicted and ground-truth tool names as sets. Acc-Args additionally requires the tool arguments to be correct: a sample with any incorrect argument counts as incorrect, and missed proactive actions are penalised (Yang et al., 2025b).

Statistical analysis. Because all configurations are evaluated on the same 400 test samples, we compare them with a paired bootstrap over test samples (10,000 resamples), recomputing each metric on every resample, and report the difference with its 95% confidence interval. Where several training seeds are available, each configuration's metric is averaged over its seeds within every resample. For claims of no difference we use the two one-sided tests (TOST) procedure with an equivalence margin of ± 0.03, chosen before the analysis as roughly the seed-to-seed spread plus evaluation noise; equivalence at $\alpha = 0.05$ holds if the 90% confidence interval lies inside the margin. To assess whether the ranking of methods reproduces, we compute Kendall's τ between our and the reported F1-scores. All comparisons are listed in Appendix D.

Costs. Training cost is reported as duration and peak and average GPU memory. Inference cost is reported as peak and average GPU memory (GB), average, minimum and maximum batch latency (s) and throughput (samples/s), with a batch size of 1.

6 Results

We first report the reproduction of the original results (Section 6.1) and then the efficiency experiments (Section 6.2).

6.1 Reproduction of the Original Results

Table 1 reports all methods with the released configuration, and Table 2 compares the reproduced ICL-All and *ContextAgent* results with the values reported by Yang et al. (2025b).

Table 1: Reproduction on *ContextAgentBench* (400 test samples) with the released configuration (*ContextAgent*: LoRA $r = 8$, learning rate 5×10^{-5}; mean ± standard deviation over three training seeds).

		Proactive Predictions				Tool Calling			
Model	Method	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	Vanilla ICL	0.730	0.225	0.045	2.066	0.569	0.514	0.525	0.246
	CoT	0.713	0.242	0.045	2.007	0.568	0.503	0.519	0.259
	ICL-P	0.777	0.165	0.058	2.007	0.633	0.544	0.567	0.374
	ICL-All	0.755	0.212	**0.033**	1.935	0.602	0.545	0.558	0.294
	ContextAgent	**0.915±0.007**	**0.044±0.011**	0.041±0.006	**1.124±0.033**	**0.764±0.006**	**0.745±0.006**	**0.740±0.005**	**0.593±0.008**
DeepSeek-R1-7B	Vanilla ICL	0.627	0.315	0.058	2.711	0.470	0.445	0.451	0.129
	CoT	0.632	0.302	0.065	2.645	0.471	0.450	0.455	0.153
	ICL-P	0.710	0.212	0.077	2.387	0.514	0.467	0.479	0.152
	ICL-All	0.720	0.237	**0.043**	2.393	0.515	0.488	0.494	0.173
	ContextAgent	**0.855±0.019**	**0.056±0.008**	0.089±0.016	**1.431±0.084**	**0.613±0.013**	**0.608±0.019**	**0.596±0.015**	**0.365±0.008**
Qwen2.5-7B-Ins	Vanilla ICL	0.800	0.165	0.035	2.079	0.583	0.554	0.558	0.298
	CoT	0.782	0.193	**0.025**	2.095	0.563	0.530	0.538	0.249
	ICL-P	0.880	0.072	0.048	1.726	0.568	0.546	0.548	0.283
	ICL-All	0.865	0.095	0.040	1.741	0.631	0.570	0.583	0.370
	ContextAgent	**0.912±0.010**	**0.041±0.008**	0.048±0.004	**1.129±0.052**	**0.732±0.006**	**0.716±0.010**	**0.707±0.008**	**0.546±0.016**

Table 2: Reported (Yang et al., 2025b) versus reproduced results. *Code*: released configuration (learning rate 5×10^{-5}); *paper*: learning rate 10^{-4} as stated in the original paper. Both use LoRA $r = 8$ on all linear layers. ContextAgent (ours) rows report the mean ± standard deviation over three training seeds (42, 1, 2).

		Proactive Predictions				Tool Calling			
Model	Method	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	ICL-All (reported)	0.757	0.229	0.012	1.872	0.631	0.565	0.582	0.270
	ICL-All (ours)	0.755	0.212	0.033	1.935	0.602	0.545	0.558	0.294
	ContextAgent (reported)	0.874	0.030	0.095	1.408	0.660	0.627	0.626	0.448
	ContextAgent (ours, code)	0.915±0.007	0.044±0.011	0.041±0.006	1.124±0.033	0.764±0.006	0.745±0.006	0.740±0.005	0.593±0.008
	ContextAgent (ours, paper)	0.927±0.016	0.040±0.007	0.033±0.009	1.024±0.077	0.800±0.008	0.785±0.006	0.777±0.005	0.664±0.004
DeepSeek-R1-7B	ICL-All (reported)	0.704	0.268	0.027	2.540	0.545	0.510	0.518	0.103
	ICL-All (ours)	0.720	0.237	0.043	2.393	0.515	0.488	0.494	0.173
	ContextAgent (reported)	0.888	0.027	0.085	1.319	0.676	0.648	0.647	0.468
	ContextAgent (ours, code)	0.855±0.019	0.056±0.008	0.089±0.016	1.431±0.084	0.613±0.013	0.608±0.019	0.596±0.015	0.365±0.008
	ContextAgent (ours, paper)	0.888±0.010	0.048±0.009	0.063±0.011	1.238±0.031	0.698±0.008	0.699±0.011	0.685±0.006	0.502±0.003
Qwen2.5-7B-Ins	ICL-All (reported)	0.867	0.088	0.044	1.721	0.635	0.577	0.591	0.301
	ICL-All (ours)	0.865	0.095	0.040	1.741	0.631	0.570	0.583	0.370
	ContextAgent (reported)	0.894	0.013	0.091	1.264	0.672	0.644	0.645	0.459
	ContextAgent (ours, code)	0.912±0.010	0.041±0.008	0.048±0.004	1.129±0.052	0.732±0.006	0.716±0.010	0.707±0.008	0.546±0.016
	ContextAgent (ours, paper)	0.925±0.008	0.044±0.009	0.031±0.004	1.060±0.017	0.765±0.006	0.747±0.002	0.740±0.005	0.598±0.024

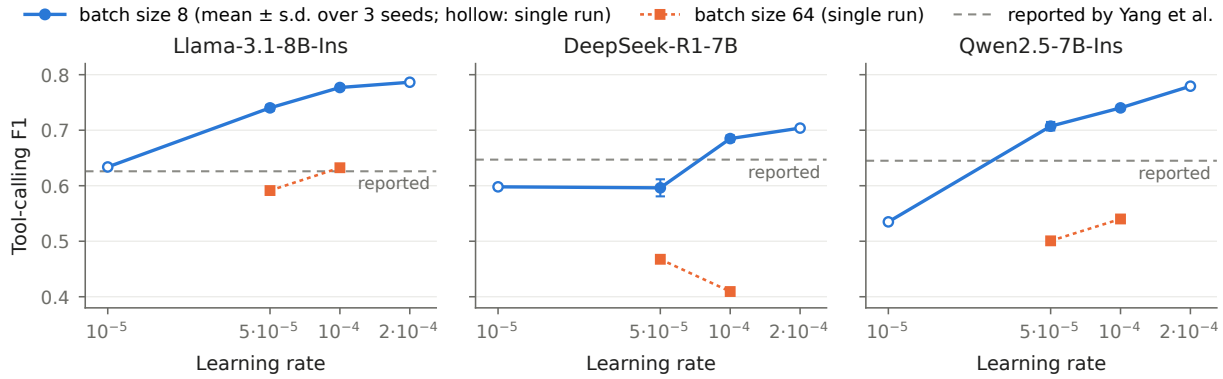


Figure 1: Tool-calling F1-score versus learning rate (LoRA $r = 8$, all linear layers). Blue: effective batch size 8 (filled: mean ± standard deviation over three seeds; hollow: single run). Orange: effective batch size 64, as with eight data-parallel GPUs (single runs). Dashed line: score reported by Yang et al. (2025b).

C1: fine-tuning outperforms in-context learning (reproduced). For all three models, *ContextAgent* achieves the highest Acc-P, the lowest MD and RMSE, and the best tool-calling scores of all methods (Table 1). The only exception is the FD, where some ICL baselines are lower, as in the original paper. For Llama and Qwen the margin over ICL-All is considerably larger than reported: tool-calling F1 improves by 0.182, 0.102 and 0.124 for Llama, DeepSeek and Qwen with the released configuration, against 0.044, 0.129 and 0.054 in the original paper.

C2: personas and thought traces help in-context learning (reproduced). ICL-All improves over Vanilla ICL in Acc-P, F1-score and Acc-Args for all three models. As in the original paper, adding thought traces alone (CoT) does not consistently help. The ranking of the five methods by tool-calling F1-score also agrees with the original paper: Kendall's τ between our and the reported values is 0.80 for each model and 0.82 over all 15 method–model pairs ($p < 0.001$).

C3: reported absolute scores (partially reproduced). The in-context baselines reproduce closely: Acc-P is within 0.02 and F1-score within 0.025 of the reported ICL-All values. The fine-tuned models, however, differ substantially. With the released configuration, Llama and Qwen exceed the reported tool-calling F1 by 0.114 and 0.062, while DeepSeek falls 0.051 below it. With the learning rate stated in the paper, all three models exceed the reported values, by 0.151 (Llama), 0.038 (DeepSeek) and 0.095 (Qwen) in F1 and by 0.034–0.216 in Acc-Args. These are means over three training seeds; the spread over seeds (standard deviation of the F1-score at most 0.015) is small compared with these differences (Figure 1). The error profile also differs: the reported models have a low MD and a high FD (0.085–0.095), whereas our Llama and Qwen models are balanced (MD 0.040–0.044, FD 0.031–0.048); only DeepSeek at the released learning rate has a similarly high FD (0.089).

To test whether eight-GPU data-parallel training explains the gap to the reported scores, we trained each model with an effective batch size of 64 (≈ 50 optimiser steps, against ≈ 375 at batch size 8; Section 5.3, Table 3 and the orange series in Figure 1). For Llama-3.1-8B-Instruct, batch size 64 with the paper's learning rate (10^{-4}) reproduces the reported tool-calling F1 closely (0.632 versus 0.626 reported; Acc-Args 0.409 versus 0.448). For DeepSeek-R1-7B and Qwen2.5-7B-Instruct, however, batch size 64 lowers tool-calling F1 well below the reported values (0.409 and 0.540 at 10^{-4}, against 0.647 and 0.645 reported); their reported scores instead lie between our batch-64 and batch-8 results (DeepSeek: between our batch-8 runs at 5×10^{-5} and 10^{-4}, 0.596 and 0.685; Qwen: between batch 64 at 10^{-4}, 0.540, and batch 8 at 5×10^{-5}, 0.707). Eight-GPU data-parallel training is therefore a plausible explanation for the reported Llama results, but not for the other two models. In either case, the qualitative conclusions of the original paper hold. At batch size 8, the released code yields a stronger *ContextAgent* than reported for Llama and Qwen, and for all three models with the paper's learning rate.

An analysis of the raw outputs shows why DeepSeek fails with an effective batch size of 64. With about 50 optimiser steps at the released learning rate (5×10^{-5}), it never adopts the trained output format: none of its 400 outputs contains the required proactive-score and tool-calling fields; 72% answer in the free-form style of the base model and 28% never close the reasoning trace. At 10^{-4}, the format is largely learned (96% of outputs are parseable), but the decisions remain poor: false alarms (FD) rise from 6% to 18%, 10% of the outputs contain missing or malformed tool calls (batch size 8: 1%), and only 2% of all samples receive exactly the correct tool set (batch size 8: 15%). With 375 optimiser steps at batch size 8, these failures are rare. Llama, in contrast, learns the output format even at batch size 64 (at most 3% unparseable outputs); its lower scores come from less accurate tool selection (36–38% wrong or incomplete tool names). Qwen shows the same failure as DeepSeek in a milder form: at batch size 64 and 5×10^{-5}, 48% of its outputs contain no proactive score (batch size 8: 0%), which lowers its tool-calling F1 to 0.501. At 10^{-4} it learns the format (all outputs parseable), but its decisions remain worse than at batch size 8: false alarms rise from 3% to 8%, missed detections from 4% to 8%, and only 4% of the samples receive exactly the correct tool set (batch size 8: 14%). With about 50 optimiser steps, the released learning rate is therefore too low for two of the three models to learn the output format reliably, and even at 10^{-4} the decisions of DeepSeek and Qwen remain clearly worse than with 375 steps; DeepSeek, whose base model produces long free-form reasoning, is affected most.

Table 3: Effect of effective batch size 64 (≈ 50 optimiser steps, matching eight-GPU data-parallel training) versus batch size 8 (≈ 375 steps) and the values reported by Yang et al. (2025b). Batch-64 rows are single runs; batch-8 rows are the mean over three seeds (Table 2). †No output contains the required proactive score; the evaluation then treats every sample as "no assistance needed", so these scores equal those of a model that never acts (cf. structured magnitude pruning, Table 12).

		Proactive Predictions				Tool Calling			
Model	**Setting**	**Acc-P↑**	**MD↓**	**FD↓**	**RMSE↓**	**Precision↑**	**Recall↑**	**F1-score↑**	**Acc-Args↑**
Llama-3.1-8B-Ins	Reported	0.874	0.030	0.095	1.408	0.660	0.627	0.626	0.448
	Batch 64, 5×10^{-5}	0.885	0.055	0.060	1.353	0.636	0.593	0.591	0.263
	Batch 64, 10^{-4}	0.850	0.087	0.062	1.400	0.648	0.647	0.632	0.409
	Batch 8, 10^{-4} (ours)	0.927	0.040	0.033	1.024	0.800	0.785	0.777	0.664
DeepSeek-R1-7B	Reported	0.888	0.027	0.085	1.319	0.676	0.648	0.647	0.468
	Batch 64, 5×10^{-5}†	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000
	Batch 64, 10^{-4}	0.750	0.068	0.182	1.933	0.419	0.439	0.409	0.091
	Batch 8, 10^{-4} (ours)	0.888	0.048	0.063	1.238	0.698	0.699	0.685	0.502
Qwen2.5-7B-Ins	Reported	0.894	0.013	0.091	1.264	0.672	0.644	0.645	0.459
	Batch 64, 5×10^{-5}	0.655	0.285	0.060	3.013	0.535	0.494	0.501	0.222
	Batch 64, 10^{-4}	0.840	0.078	0.083	1.404	0.568	0.543	0.540	0.284
	Batch 8, 10^{-4} (ours)	0.925	0.044	0.031	1.060	0.765	0.747	0.740	0.598

Cost of the reproduced models. Table 14 in Appendix B reports the training cost of the reproduced *ContextAgent* models: all three peak at about 21 GB of GPU memory and train in 45–58 minutes. Table 4 reports their inference cost, which serves as the reference for the efficiency experiments.

Table 4: Inference cost of the reproduced ContextAgent models (16-bit, released configuration). Measured on the runs with the exact released configuration (Section 5.3).

		Costs					
Method	Model	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
ContextAgent	Llama-3.1-8B-Ins	15.6	15.6	9.0	4.0	21.7	0.112
	DeepSeek-R1-7B	14.8	14.8	8.7	2.4	23.1	0.115
	Qwen2.5-7B-Ins	14.8	14.8	8.3	1.7	22.6	0.121

The three models need 14.8–15.6 GB at inference and process one sample in 8.3–9.0 s on average.

6.2 Efficiency under Resource Constraints

All experiments in this section use the *ContextAgent* configuration with thought traces and personas. Unless stated otherwise, the LoRA configuration is the released default: rank $r = 8$, all linear layers, learning rate 5×10^{-5}. Because all sweeps are evaluated on the test split (Section 5.3), they are a sensitivity analysis rather than a model selection procedure.

6.2.1 LoRA Configuration

Table 5: Effect of the LoRA rank r (learning rate 5×10^{-5}, all linear layers; $r = 8$ is the released default). All rows are single runs (seed 42); the $r = 8$ row is the seed-42 run of Table 1.

		Proactive Predictions				Tool Calling			
Model	Rank	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	8	0.917	0.045	0.037	1.129	0.762	0.739	0.735	0.595
	16	**0.938**	**0.030**	**0.033**	1.033	**0.803**	**0.795**	0.782	0.658
	32	0.935	0.033	**0.033**	1.004	0.801	0.784	0.777	0.683
	64	0.935	0.033	**0.033**	**0.937**	0.802	0.793	**0.783**	**0.685**
DeepSeek-R1-7B	8	0.873	0.048	0.080	1.365	0.625	0.622	0.609	0.368
	16	0.880	0.060	0.060	1.295	0.657	0.661	0.646	0.433
	32	0.897	0.045	**0.058**	1.278	0.694	0.697	0.678	**0.490**
	64	**0.908**	**0.033**	0.060	**1.160**	**0.701**	**0.710**	**0.690**	0.489
Qwen2.5-7B-Ins	8	0.915	0.035	0.050	1.137	0.734	0.719	0.708	0.551
	16	0.922	0.037	0.040	1.075	0.774	0.742	0.739	0.618
	32	**0.943**	0.033	**0.025**	**0.927**	**0.798**	**0.782**	**0.770**	0.631
	64	0.935	**0.030**	0.035	0.987	0.795	0.771	0.769	**0.649**

Table 5 presents the effect of LoRA rank on task performance across all three models, with all other hyperparameters fixed to their baseline values. For `Llama-3.1-8B-Instruct`, the F1-score rises from 0.735 at $r = 8$ to 0.782 at $r = 16$ and then plateaus (0.777 at $r = 32$, 0.783 at $r = 64$). `Qwen2.5-7B-Instruct` improves up to $r = 32$, with F1-scores of 0.708, 0.739, 0.770 and 0.769. *DeepSeek-R1-7B* improves at every rank, from 0.609 at $r = 8$ to 0.690 at $r = 64$, although the step from $r = 32$ (0.678) to $r = 64$ is within run-to-run uncertainty.

Table 6: LoRA target modules ($r = 8$, learning rate 5×10^{-5}).

		Proactive Predictions				Tool Calling			
Model	Target modules	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	All linear	**0.917**	0.045	**0.037**	**1.129**	**0.762**	**0.739**	**0.735**	**0.595**
	Attention only	0.890	0.065	0.045	1.262	0.723	0.707	0.699	0.512
	MLP only	0.905	**0.037**	0.058	1.224	0.731	0.714	0.706	0.556
DeepSeek-R1-7B	All linear	**0.873**	**0.048**	0.080	**1.365**	**0.625**	**0.622**	**0.609**	**0.368**
	Attention only	0.823	0.058	0.120	1.527	0.516	0.519	0.506	0.231
	MLP only	**0.873**	0.050	**0.078**	1.384	0.613	0.599	0.594	0.361
Qwen2.5-7B-Ins	All linear	**0.915**	**0.035**	0.050	1.137	**0.734**	**0.719**	0.708	0.551
	Attention only	0.877	0.062	0.060	1.312	0.643	0.628	0.620	0.412
	MLP only	**0.915**	0.048	**0.037**	**1.124**	0.732	0.715	**0.709**	**0.562**

Table 6 reports the effect of restricting LoRA adapters to specific architectural components, evaluated at the baseline rank and learning rate. For `Llama-3.1-8B-Instruct`, adapting all linear layers gives the best results (F1-score 0.735), and adapting only the attention projections gives the worst (0.699). For `Qwen2.5-7B-Instruct`, adapting only the MLP layers is as good as adapting all linear layers (0.709 versus 0.708), while attention-only adaptation is clearly worse (0.620). *DeepSeek-R1-7B* shows the same pattern: adapting only the MLP layers comes close to adapting all linear layers (F1-score 0.594 versus 0.609), whereas attention-only adaptation loses 0.10 (0.506) and raises false detections to 0.120.

Table 7: Learning rate (LoRA $r = 8$, all linear layers). Scores recomputed from the released prediction files. The 5×10^{-5} and 10^{-4} rows show the mean ± standard deviation over three training seeds (42, 1, 2); the 10^{-5} and 2×10^{-4} rows are single runs.

		Proactive Predictions				Tool Calling			
Model	LR	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	1.0×10^{-5}	0.885	**0.035**	0.080	1.253	0.668	0.638	0.634	0.415
	5.0×10^{-5} (code)	0.915±0.007	0.044±0.011	0.041±0.006	1.124±0.033	0.764±0.006	0.745±0.006	0.740±0.005	0.593±0.008
	1.0×10^{-4} (paper)	0.927±0.016	0.040±0.007	0.033±0.009	1.024±0.077	0.800±0.008	0.785±0.006	0.777±0.005	0.664±0.004
	2.0×10^{-4}	**0.930**	0.040	**0.030**	**0.999**	**0.805**	**0.796**	**0.786**	**0.685**
DeepSeek-R1-7B	1.0×10^{-5}	0.868	0.058	0.075	1.367	0.621	0.598	0.598	0.376
	5.0×10^{-5} (code)	0.855±0.019	0.056±0.008	0.089±0.016	1.431±0.084	0.613±0.013	0.608±0.019	0.596±0.015	0.365±0.008
	1.0×10^{-4} (paper)	**0.888±0.010**	**0.048±0.009**	**0.063±0.011**	**1.238±0.031**	0.698±0.008	0.699±0.011	0.685±0.006	0.502±0.003
	2.0×10^{-4}	0.882	0.052	0.065	1.273	**0.720**	**0.721**	**0.704**	**0.527**
Qwen2.5-7B-Ins	1.0×10^{-5}	0.825	0.062	0.113	1.564	0.580	0.525	0.535	0.337
	5.0×10^{-5} (code)	0.912±0.010	0.041±0.008	0.048±0.004	1.129±0.052	0.732±0.006	0.716±0.010	0.707±0.008	0.546±0.016
	1.0×10^{-4} (paper)	0.925±0.008	0.044±0.009	**0.031±0.004**	1.060±0.017	0.765±0.006	0.747±0.002	0.740±0.005	0.598±0.024
	2.0×10^{-4}	**0.932**	**0.035**	0.033	**0.955**	**0.805**	**0.784**	**0.779**	**0.660**

Table 7 and Figure 1 compare four learning rates: 10^{-5}, the released 5×10^{-5}, the paper's 10^{-4} and 2×10^{-4}. Above 5×10^{-5}, tool-calling performance increases with the learning rate for all three models. Moving from the released 5×10^{-5} to the paper's 10^{-4} raises the F1-score from 0.740 to 0.777 (Llama), 0.596 to 0.685 (DeepSeek) and 0.707 to 0.740 (Qwen), averaged over three seeds; all three differences are significant (paired bootstrap: +0.037, +0.089 and +0.033, with 95% confidence intervals excluding zero), and the seed standard deviation of the F1-score is at most 0.015. At 2×10^{-4}, all three models improve further (Llama 0.786, DeepSeek 0.704, Qwen 0.779), although the gain over 10^{-4} is small for Llama (0.009). 10^{-5} is clearly worse for Llama and Qwen (0.634 and 0.535), whereas for DeepSeek it is on par with the released value (0.598 versus 0.596). The released value is therefore not anomalous: it lies on the lower side of an increasing trend. This is consistent with the models being undertrained at the released setting, given only 600 training samples (Section 5.3). Since 2×10^{-4} is the best tested value for all three models, we cannot exclude that larger learning rates perform better still.

6.2.2 Quantisation

Table 8: Task performance under base-model quantisation (LoRA $r = 8$, learning rate 5×10^{-5}, all linear layers). All rows are single runs (seed 42); the 16-bit row is the seed-42 run of Table 1.

		Proactive Predictions				Tool Calling			
Model	Q-Bit	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	16-bit	0.917	0.045	0.037	1.129	0.762	0.739	0.735	0.595
	8-bit	0.912	0.055	0.033	1.150	0.775	0.741	0.742	0.608
	4-bit	0.892	0.037	0.070	1.256	0.736	0.712	0.708	0.577
DeepSeek-R1-7B	16-bit	0.873	0.048	0.080	1.365	0.625	0.622	0.609	0.368
	8-bit	0.865	0.065	0.070	1.414	0.631	0.631	0.615	0.373
	4-bit	0.843	0.060	0.098	1.476	0.619	0.600	0.596	0.385
Qwen2.5-7B-Ins	16-bit	0.915	0.035	0.050	1.137	0.734	0.719	0.708	0.551
	8-bit	0.915	0.050	0.035	1.161	0.753	0.722	0.720	0.542
	4-bit	0.863	0.090	0.048	1.620	0.696	0.693	0.680	0.498

Table 8 compares 16-bit LoRA, 8-bit LLM.int8 and 4-bit NF4 QLoRA, with all other settings at the released default. 8-bit quantisation does not measurably reduce task performance: the F1-score changes by +0.007 (Llama), +0.006 (DeepSeek) and +0.012 (Qwen) relative to 16-bit, and none of these differences are significant (all 95% confidence intervals include zero). 4-bit NF4 lowers the F1-score by 0.013–0.028 (Llama 0.735 to 0.708, DeepSeek 0.609 to 0.596, Qwen 0.708 to 0.680); these differences are also not significant individually. The only significant effect is a lower Acc-P for 4-bit Qwen (−0.052, 95% CI [−0.090, −0.015]). With single runs, the evaluation is not precise enough to establish equivalence within ±0.03 (TOST), so we report these

results as no detectable difference rather than as equivalence. These results agree with what is known for QLoRA on other tasks (Dettmers et al., 2023): base-model quantisation retains most of the fine-tuned performance, and we include them as a supporting cost–performance reference for the reproduced models rather than as a new finding.

The training resource requirements corresponding to these configurations are reported in Table 15 in Appendix B. For training, 4-bit quantisation reduces peak GPU memory by 47% for Llama (21.10 to 11.28 GB), but only by 25% for Qwen (21.37 to 15.95 GB) and 10% for DeepSeek (21.12 to 18.91 GB). Quantised training is also slower, by 9–29% at 4-bit and 23–28% at 8-bit.

Table 9: Inference cost under base-model quantisation (configuration as in Table 8). The 16-bit rows repeat Table 4.

		Costs					
Model	Q-Bit	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
LLaMA-3.1-8B-Ins	16-bit	15.6	15.6	9.0	4.0	21.7	0.112
	8-bit	9.3	9.3	17.4	7.1	45.8	0.057
	4-bit	8.2	8.1	11.4	4.9	25.2	0.088
DeepSeek-R1-7B	16-bit	14.8	14.8	8.7	2.4	23.1	0.115
	8-bit	11.0	9.9	18.2	6.8	55.0	0.055
	4-bit	7.7	7.7	10.2	4.4	25.3	0.098
Qwen2.5-7B-Ins	16-bit	14.8	14.8	8.3	1.7	22.6	0.121
	8-bit	9.6	9.2	18.5	3.5	45.6	0.054
	4-bit	7.7	7.7	11.8	0.7	81.2	0.085

At inference (Table 9), peak GPU memory decreases with the bit width for all three models; for Llama it falls from 15.6 GB at 16-bit to 9.3 GB at 8-bit and 8.2 GB at 4-bit (40% and 47% reductions). Latency does not follow the same ordering. With the bitsandbytes kernels used here, LLM.int8 roughly doubles the average time per batch compared to 16-bit, while 4-bit NF4 is 17–42% slower than 16-bit but clearly faster than 8-bit.

6.2.3 Combined Configuration

Table 10: Best configuration per model found on the test split (optimistic; see Section 5.3), at 16-bit and 4-bit. Mean ± standard deviation over three training seeds (42, 1, 2).

		Proactive Predictions				Tool Calling			
Model	Config	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
LLaMA-3.1-8B-Ins	$r=32, lr=1.0\times10^{-4}$, all, 16-bit	0.928±0.003	**0.033±0.007**	0.039±0.009	0.998±0.014	0.788±0.005	0.784±0.017	0.771±0.011	0.657±0.019
	$r=32, lr=1.0\times10^{-4}$, all, 4-bit	**0.933±0.010**	0.038±0.005	**0.028±0.006**	**0.981±0.090**	**0.807±0.016**	**0.795±0.009**	**0.787±0.012**	**0.664±0.018**
DeepSeek-R1-7B	$r=64, lr=1.0\times10^{-4}$, all, 16-bit	**0.892±0.016**	**0.044±0.005**	0.064±0.016	**1.225±0.074**	**0.717±0.020**	0.712±0.024	0.698±0.022	0.516±0.020
	$r=64, lr=1.0\times10^{-4}$, all, 4-bit	0.888±0.011	0.049±0.010	**0.062±0.005**	1.259±0.042	0.716±0.011	**0.729±0.018**	**0.708±0.012**	**0.535±0.014**
Qwen2.5-7B-Ins	$r=32, lr=1.0\times10^{-4}$, all, 16-bit	0.925±0.010	**0.031±0.003**	0.044±0.008	1.084±0.062	0.777±0.011	0.761±0.018	0.753±0.014	0.624±0.028
	$r=32, lr=1.0\times10^{-4}$, all, 4-bit	**0.928±0.006**	0.039±0.006	**0.033±0.005**	**1.042±0.030**	**0.788±0.005**	**0.776±0.008**	**0.769±0.004**	**0.648±0.011**

Table 10 combines the best settings found above for each model. For Llama and Qwen this is $r = 32$, learning rate 10^{-4} and all linear layers; for DeepSeek it is $r = 64$, learning rate 10^{-4} and all linear layers, the best DeepSeek configuration in our sweeps. All six rows are averaged over three training seeds. Once the learning rate is 10^{-4}, the higher rank adds no significant gain for any model: compared with $r = 8$ at 10^{-4} (three seeds each), the F1-score changes by -0.006 for Llama (0.771 versus 0.777, 95% CI $[-0.028, 0.015]$), $+0.013$ for DeepSeek (0.698 versus 0.685, $[-0.012, 0.037]$) and $+0.013$ for Qwen (0.753 versus 0.740, $[-0.011, 0.037]$). The improvement over the released default therefore comes from the learning rate. The 4-bit version of each combined configuration scores slightly higher than the 16-bit version (Llama 0.787 versus 0.771, DeepSeek 0.708 versus 0.698, Qwen 0.769 versus 0.753), but none of these differences is significant (Llama $+0.016$, 95% CI $[-0.003, 0.035]$; DeepSeek $+0.010$, $[-0.010, 0.031]$; Qwen $+0.016$, $[-0.004, 0.035]$). With three seeds,

the evaluation is precise enough to show equivalence within ±0.03 for DeepSeek's F1-score and for Acc-P of all three models. 4-bit models with a suitable configuration therefore match or exceed the released 16-bit configuration (0.740, 0.596 and 0.707) at about half the inference memory.

Training costs for these configurations are in Table 16 (Appendix B). At 4-bit, peak training memory falls from 21.63 to 12.24 GB for Llama, from 21.22 to 11.54 GB for DeepSeek and from 21.91 to 15.26 GB for Qwen, with training time increasing by 10% for Llama, 25% for DeepSeek and less than 1% for Qwen.

Table 11: Inference cost of the combined configurations.

		Costs					
Model	Config	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
Llama-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	15.6	15.6	**9.1**	**4.2**	**21.7**	**0.109**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**8.3**	**8.3**	13.9	5.5	33.9	0.072
DeepSeek-R1-7B	$r = 64, lr = 1.0 \times 10^{-4}$, all, 16-bit	14.8	14.8	8.5	3.5	20.3	0.117
	$r = 64, lr = 1.0 \times 10^{-4}$, all, 4-bit	7.8	7.8	8.6	0.5	18.8	0.116
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	14.8	14.8	**8.5**	**3.5**	**19.5**	**0.118**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**7.7**	**7.7**	11.3	4.4	29.2	0.089

At inference (Table 11), 4-bit quantisation reduces peak GPU memory by 47–48% for all three models (Llama 15.6 to 8.3 GB, DeepSeek 14.8 to 7.8 GB, Qwen 14.8 to 7.7 GB). For Llama and Qwen this comes with a 53% and 33% increase in average batch time; for DeepSeek the average batch time is almost unchanged (8.5 versus 8.6 s).

6.2.4 Pruning

Pruning experiments use `Llama-3.1-8B-Instruct` with the released configuration; all results are after LoRA-based recovery fine-tuning. The dense reference is the seed-42 *ContextAgent* run (the $r = 8$ row of Table 1; F1-score 0.735, Acc-Args 0.595).

Table 12: Unstructured and structured pruning of `Llama-3.1-8B-Instruct` after recovery fine-tuning. For unstructured pruning, the binary pruning mask is re-applied after merging the recovery adapter, ensuring the evaluated weights have the exact stated sparsity; rows with three evaluations report the mean. ‡Mask-aware recovery: the LoRA update is restricted to the unpruned weights (Section 5.5; single run). Rows marked 10^{-4} use mask-aware recovery with learning rate 10^{-4} and the stated rank instead of the released recovery configuration ($r = 8$, 5×10^{-5}); their dense reference is the seed-42 run at 10^{-4}, $r = 8$ (F1-score 0.781, Acc-Args 0.661). Dense reference: F1-score 0.735, Acc-Args 0.595 (seed-42 run, $r = 8$ row of Table 5).

			Proactive Predictions				Tool Calling			
Structure	Pruning	%	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Unstructured	Wanda	10	0.897	0.033	0.070	1.188	0.752	0.710	0.715	0.598
		30	0.920	0.048	0.033	1.108	0.764	0.692	0.708	0.603
		50	0.886	0.067	0.048	1.276	0.693	0.618	0.634	0.459
		50‡	0.895	0.072	0.033	1.217	0.724	0.718	0.706	0.521
		50‡, 10^{-4}, r=8	0.912	0.058	**0.030**	1.124	0.741	0.725	0.717	0.566
		50‡, 10^{-4}, r=32	**0.930**	0.035	0.035	**1.031**	**0.787**	**0.782**	**0.771**	**0.642**
	Magnitude	10	0.915	0.043	0.043	1.134	0.740	0.699	0.705	0.577
		30	0.908	**0.030**	0.063	1.180	0.727	0.658	0.673	0.577
		50	0.515	0.480	0.005	4.629	0.449	0.445	0.446	0.021
		50‡	0.887	0.050	0.062	1.279	0.697	0.695	0.679	0.481
Structured	Activation	10	**0.870**	**0.083**	**0.048**	**1.371**	**0.652**	**0.651**	**0.637**	**0.410**
		30	0.718	0.172	0.110	1.944	0.493	0.490	0.481	0.236
		50	0.522	0.315	0.163	2.516	0.315	0.309	0.310	0.047
	Magnitude	10	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000
		30	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000
		50	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000

Table 12 shows that unstructured pruning preserves performance much better than structured pruning. With the binary mask re-applied after merging the recovery adapter, WANDA preserves performance well at 10% and 30% sparsity: the F1-score remains at 0.715 and 0.708 (dense reference: 0.735), and Acc-Args is maintained at 0.598 and 0.603 (dense: 0.595). At 50% sparsity, however, enforcing strict zeros degrades performance noticeably, with the F1-score dropping by 0.10 to 0.634 and Acc-Args to 0.459. Magnitude pruning, which ignores activations, degrades faster. At 10% sparsity it is still close to the dense model (F1-score 0.705; difference -0.030, 95% CI $[-0.063, 0.002]$), but at 30% it already loses 0.06 (0.673, CI $[-0.100, -0.024]$), and at 50% the model largely loses the output format: 90% of its outputs contain no proactive score, so Acc-P falls to 0.515, MD rises to 0.480 and Acc-Args to 0.021. At the same 50% sparsity, WANDA keeps an F1-score of 0.634, which confirms that activation-aware importance matters at high sparsity (Sun et al., 2024).

Most of this loss is caused by the recovery procedure rather than by the sparsity itself. With mask-aware recovery, the LoRA update is trained only on the weights that remain after pruning, so nothing of it is discarded when the mask is applied. This improves every setting significantly (paired bootstrap, all 95% confidence intervals above zero): Wanda at 50% sparsity rises from 0.634 to 0.706 (+0.072, 95% CI $[0.041, 0.104]$) and is no longer significantly below the dense model in F1-score (-0.029, $[-0.063, 0.005]$), although its Acc-Args remains lower (0.521 versus 0.595); magnitude pruning at 50%, which previously lost the output format, rises from 0.446 to 0.679 (+0.233, $[0.186, 0.280]$). The error cascade (Section E.1) shows why: the share of outputs without a parseable proactive score falls from 92.5% to 0.5% for 50% magnitude pruning and from 34% to 0% for Wanda 2:4, and the remaining errors are tool-selection errors, as for the dense model. The price is a three times longer recovery training (about 8,480 s instead of 2,770 s at a peak of 19.8 GB), because the masked update is materialised as a full matrix in every forward pass; inference cost is unchanged.Mask-aware recovery was first run with the released recovery configuration ($r = 8$, 5×10^{-5}), which is itself sub-optimal for the dense model (Section 6.2.1). Repeating it with the paper's learning rate of 10^{-4} (Table 12 and Table 18) closes most of the remaining gap. At $r = 8$ the F1-score rises to 0.717 for Wanda at 50%, 0.711 for 2:4 and 0.745 for 4:8; for 2:4 and 4:8 this is a significant gain over mask-aware recovery at 5×10^{-5} (+0.062, 95% CI $[0.024, 0.100]$, and +0.079, $[0.045, 0.114]$), for 50% it is not (+0.011, $[-0.018, 0.041]$). With $r = 32$ the three settings reach 0.771, 0.744 and 0.761. Compared with the dense model trained at the same learning rate, Wanda at 50% (-0.010, $[-0.041, 0.019]$) and at 4:8 (-0.020, $[-0.051, 0.011]$) show no significant difference in F1-score, and neither does Wanda at 50% in *Acc-Args* (0.642 versus 0.661; -0.020, $[-0.067, 0.027]$); only the 2:4 pattern remains significantly below (-0.037, $[-0.070, -0.005]$). These $r = 32$ runs combine a higher rank with mask-aware recovery, but rank alone makes no significant difference for the dense model at 10^{-4} (Table 10). At 50% sparsity, a pruned model with mask-aware recovery at the paper's learning rate therefore performs on par with the dense model fine-tuned in the same way. The training cost of mask-aware recovery is unchanged by the learning rate (8,298–8,442 s, peak 19.76 GB at $r = 8$ and 20.36 GB at $r = 32$; Tables 17 and 19). These are single runs.

Structured activation-based pruning loses 0.10 in F1-score already at 10% (0.637) and collapses at 50% (0.310). Structured magnitude pruning yields identical results at all three ratios, with MD = 0.530 and FD = 0.000: the model never predicts that assistance is needed. Its Acc-P and F1-score of 0.468 therefore correspond to the share of non-proactive samples, whose empty tool set is trivially matched. This is a degenerate solution rather than a gradual degradation.

Training costs (Table 17, Appendix B) show that unstructured pruning does not reduce recovery training time or memory, whereas 50% structured pruning reduces training time by about 48% and peak memory by 43% compared to 10% structured pruning.

Table 13: Inference cost after pruning and recovery. ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

			Costs					
Structure	Method	%	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
Unstructured	Wanda	10	15.6	15.6	8.7	3.8	19.3	0.1154
		30	15.6	15.6	8.1	3.9	20.1	0.1233
		50	15.7	15.7	8.2	3.9	68.0	0.1227
		50‡	15.6	15.6	9.6	4.1	31.5	0.1043
		50‡, 10^{-4}, r=8	15.6	15.6	9.2	2.9	22.2	0.1086
		50‡, 10^{-4}, r=32	15.6	15.6	9.8	4.5	23.4	0.1018
	Magnitude	10	15.6	15.6	8.7	3.8	23.5	0.1148
		30	15.6	15.6	8.1	3.9	17.1	0.1227
		50	15.7	15.7	10.2	2.9	68.9	0.0981
		50‡	15.7	15.6	10.0	4.3	61.0	0.1000
Structured	Activation	10	13.2	13.2	8.2	3.7	17.8	0.1220
		30	8.9	8.9	5.4	2.3	13.3	0.1850
		50	5.7	5.6	5.3	1.5	32.1	0.1883
	Magnitude	10	13.3	13.3	34.8	4.6	62.0	0.0287
		30	9.0	9.0	43.7	42.6	47.6	0.0229
		50	5.7	5.6	31.4	28.2	41.4	0.0318

At inference (Table 13), unstructured pruning leaves memory and latency unchanged because the pruned weights are stored and multiplied densely; this also holds for mask-aware recovery at 10^{-4} (15.6 GB, 9.2–9.8 s per sample). Structured magnitude pruning is 3.5–5 times *slower* than the dense model despite its smaller size. This slowdown coincides with degenerate generation: for the 10%, 30% and 50% settings, none of the 400 outputs contains a proactive score; the outputs consist of repeated fragments of quotes, braces and tool-schema words. Re-tokenising the saved outputs gives on average 463, 860 and 969 tokens per sample, against 132 for the dense model; for degenerate text this is a lower bound, because runs of whitespace and punctuation merge into fewer tokens than were generated. Calibrating against the per-sample latencies of the activation-pruned model of the same size, whose latency grows linearly with output length ($r \geq 0.999$), shows that all 400 samples at 30% and most samples at 50% (implied mean length 1,024 tokens) run until the limit of 1,024 new tokens, as do about one third (130) at 10%. The slowdown therefore comes from generating up to eight times as many tokens per sample, not from slower computation per token. The dense model and the activation-pruned models stay far below the limit (on average 127–168 tokens; only 15 of the 400 samples at 50% activation pruning reach it).

7 Discussion

7.1 What Reproduces and What Does Not

The qualitative claims of *ContextAgent* hold: fine-tuning with persona context and thought traces outperforms in-context learning for all three base models (C1), and personas plus thought traces improve in-context learning (C2). The in-context baselines also reproduce numerically. The fine-tuned results do not: with the released code, two of three models score clearly higher than reported, and with the learning rate stated in the paper all three do (C3).

Two under-specified details make this difference hard to attribute. First, the paper and the released configuration disagree on the learning rate (10^{-4} versus 5×10^{-5}), and the choice between these two values alone changes tool-calling F1 by 0.03–0.09 for the same model (Table 7). Second, the paper reports eight GPUs, but not whether training was data-parallel or what the resulting effective batch size was. With only 600 training samples, an eight-fold difference in batch size changes the number of optimiser steps from about 375 to about 50, which may leave the original models undertrained. Our batch-64 runs support this for Llama, whose reported scores they reproduce (0.632 versus 0.626), but not for DeepSeek and Qwen, which fall well below their reported values at batch size 64 (Table 3); for these two models the cause of the difference

remains open. For future users of *ContextAgentBench*, we recommend reporting the effective batch size and number of optimiser steps, and treating the learning rate as a tuned hyperparameter selected on a held-out part of the training split.

A practical consequence is that the efficiency results below are measured against a stronger baseline than the paper reports: an efficiency technique that "preserves the reported performance" may still lose a substantial part of the performance the released code can reach.

7.2 Efficiency of the Proactive Reasoner

LoRA configuration. Since *ContextAgent* already uses LoRA, the relevant question is which configuration is sufficient. At the released learning rate, a higher rank partly compensates for the low learning rate (Table 5; e.g. Llama 0.735 at $r = 8$ versus 0.782 at $r = 16$). Once the learning rate is 10^{-4}, however, rank 8 on all linear layers is sufficient for all three models: averaged over three seeds, the higher rank of the combined configurations changes the F1-score by -0.006 for Llama ($r = 32$, 0.771 versus 0.777), $+0.013$ for DeepSeek ($r = 64$, 0.698 versus 0.685) and $+0.013$ for Qwen ($r = 32$, 0.753 versus 0.740), and none of these differences is significant (all 95% confidence intervals include zero; Table 10). The gains of about 0.03–0.04 suggested by single runs were within run-to-run variation. In single runs, restricting adaptation to MLP layers helps DeepSeek at ranks 16 and 32 with 10^{-4} (F1-score 0.715 versus 0.689 and 0.686 for all linear layers) but not at $r = 64$ (0.681 versus 0.723); given the seed variation above, we do not interpret these differences further.

Pruning. With the pruning mask preserved, unstructured Wanda pruning keeps the task metrics up to 30% sparsity (no significant difference to the dense model). At 50% sparsity and with semi-structured patterns, the result depends strongly on the recovery. A recovery adapter trained without the mask and re-masked afterwards loses 0.10 F1 at 50% sparsity (95% CI $[-0.13, -0.07]$) and 0.12 and 0.27 F1 with the 4:8 and 2:4 patterns (Table 18), and magnitude pruning collapses at 50%. Restricting the recovery update to the unpruned weights reduces these losses to 0.03 (not significant), 0.07 and 0.09, and to 0.06 for magnitude pruning at 50%; for 50% magnitude pruning and the 2:4 patterns, much of the removed loss consisted of format failures (Section E.1). With the paper's learning rate and $r = 32$, mask-aware recovery brings Wanda at 50% and 4:8 to the level of the dense model trained in the same way (F1-score 0.771 and 0.761 versus 0.781; no significant difference), and 2:4 to within 0.04. Because the pruned weights are stored densely, none of these settings reduces memory or latency, and executing the 2:4 models with sparse kernels makes generation slower at batch size one (Appendix C). Structured pruning of MLP dimensions brings real memory and latency savings, but the task degrades quickly; with magnitude-based group importance, the model collapses to never predicting a need for assistance. For a proactive agent, this failure mode is particularly harmful because it is silent: the agent appears to work but never acts. Activation-based importance degrades gradually, mainly through wrong proactive decisions, which indicates that data-dependent criteria are needed for structured pruning in this setting. A further lesson concerns the recovery step: merging an unconstrained LoRA adapter into a pruned model silently restores dense weights, and re-applying the mask afterwards discards a share of the update equal to the sparsity (Appendix C); the recovery should be mask-aware from the start and use the same learning rate as fine-tuning of the dense model.

Small models. A smaller base model is the most direct way to reduce cost. Fine-tuned with LoRA at the full sequence length of 2048 tokens (learning rate 10^{-4}, all linear layers; Table 24), all three 1–1.5B models learn the output format (at most 0.3% of outputs without a proactive score), and their remaining errors are tool-selection errors, as for the 7–8B models (Appendix E.1). Over three seeds, Qwen2.5-1.5B with $r = 32$ reaches a tool-calling F1-score of 0.707 ± 0.006 and an *Acc-Args* of 0.528 ± 0.024, the level of Qwen2.5-7B with the released configuration (0.707 and 0.546), at 3.8 GB of inference memory and 4.2 s per sample instead of 14.8 GB and 8.3 s; Llama-3.2-1B reaches 0.673 ± 0.007 at 3.3 GB and DeepSeek-R1-1.5B 0.603 ± 0.009. With a 4-bit (NF4) base model, Qwen2.5-1.5B keeps its F1-score (0.710; difference to 16-bit $+0.003$, 95% CI $[-0.030, 0.036]$, single run) at only 2.2 GB, but on-the-fly dequantisation raises the time per sample to 12.1 s. At a similar memory budget, structured pruning of Llama-3.1-8B (50%, 5.7 GB) reaches an F1-score of only 0.310. The rank of the small models was chosen on the test split, so these results are optimistic, but the gap to structured pruning is far larger than the seed variation (standard deviation of the F1-score at most 0.009).

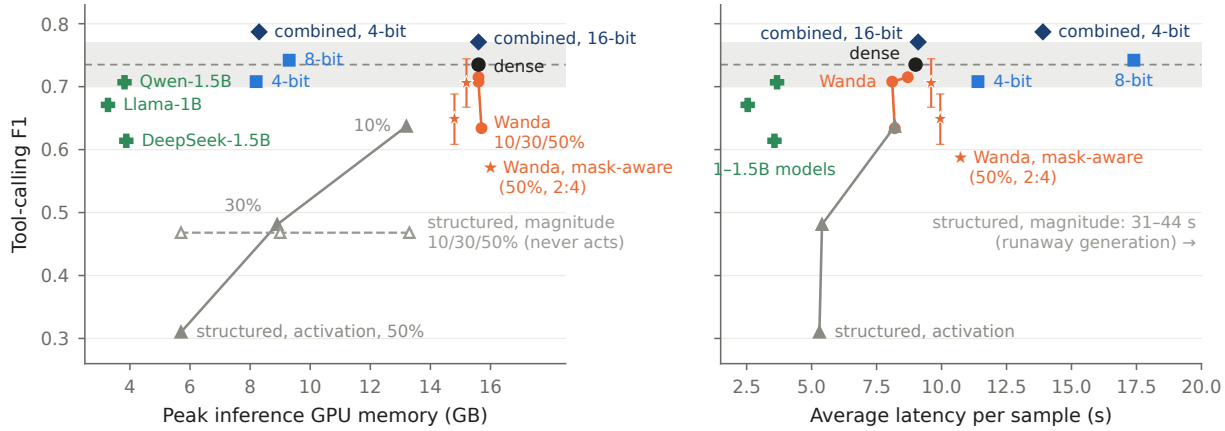


Figure 2: Cost–performance trade-off: tool-calling F1-score against peak inference GPU memory (left) and average latency per sample (right). All techniques are applied to `Llama-3.1-8B-Instruct`; the dashed line marks the dense 16-bit model ($r = 8$, seed 42) and the grey band its 95% bootstrap CI. Quantisation uses the released LoRA configuration; "combined" uses $r = 32$ and learning rate 10^{-4} (mean over three seeds). Pruning is shown with the pruning mask re-applied, at 10, 30 and 50% sparsity; pruned weights are stored densely, so unstructured pruning stays at the dense model's memory. Stars: WANDA with mask-aware recovery at 50% and 2:4 sparsity, with the released recovery configuration ($r = 8$, 5×10^{-5}) and with $r = 32$ and 10^{-4}, with 95% bootstrap CIs; they are offset horizontally for readability (all at 15.6 GB). Hollow star: the mask-aware 2:4 model executed with sparse kernels. Structured magnitude pruning (31–44 s per sample) lies outside the right panel: its outputs are degenerate and run to the 1,024-token generation limit. Green crosses: 1–1.5B models of other families fine-tuned at the full sequence length ($r = 32$, mean over three seeds; Section 7.2, Table 24); hollow cross: Qwen2.5-1.5B with a 4-bit base model. Unstructured magnitude and the remaining semi-structured settings are omitted for readability (Table 12 and Appendix C).

Cost–performance trade-off. Among the techniques tested, 4-bit quantisation of the base model with a well-chosen LoRA configuration offers the best trade-off for the 7–8B models: it matches or exceeds the released 16-bit model at about half the inference memory (Figure 2, left). A 1–1.5B model fine-tuned at the full sequence length offers an even more favourable point: Qwen2.5-1.5B matches Qwen2.5-7B at the released configuration at about a quarter of its inference memory and half its latency (3.8 GB, 4.2 s per sample, F1 0.707 ± 0.006 over three seeds), and with a 4-bit base at about a seventh of the memory (2.2 GB), at the price of three times its 16-bit latency. Mask-aware recovery at the paper's learning rate preserves the accuracy of 50% unstructured and 4:8 pruning, but with dense storage this accuracy comes without any saving: the pruned points stay at the dense model's memory and latency, and executing the 2:4 models with sparse kernels makes generation about 2.8 times slower at batch size one. Structured pruning, which loses 0.10 F1 already at 10%, moves the model to lower memory and latency only at a steep loss in F1-score. When memory must be reduced well below 8 GB, fine-tuning a 1–1.5B base model directly therefore clearly dominates structured pruning of a 7–8B model. These conclusions hold across the nine scenarios of the benchmark: neither quantisation, pruning nor the smaller model degrades some scenarios significantly more than others (permutation tests, $p \geq 0.15$), although the per-scenario samples are too small to exclude moderate differences (Appendix G).

7.3 Limitations

The released split has no validation set, so all configuration sweeps are evaluated on the test split. The "best configuration" results should therefore be interpreted as optimistic upper bounds from a sensitivity sweep rather than a tuned model selection. Apart from the released, paper and combined configurations and the 1–1.5B models with $r = 32$, which we trained with three seeds, each setting is evaluated from a single

training run ($k = 1$), so its between-run variance is not estimated. With 400 test samples, differences of a few hundredths in a single metric should not be over-interpreted.

Test-set selection and single-run evaluation. The released split has no validation set, so all configuration sweeps are evaluated on the test split. The "best configuration" results should therefore be interpreted as optimistic upper bounds from a sensitivity sweep rather than a tuned model selection. In contrast, the reproduction results for the released and paper configurations (Table 2) evaluate the specified configurations directly without hyperparameter selection. Apart from the released, paper and combined configurations and the 1–1.5B models with $r = 32$, which we trained with three seeds, each setting is evaluated from a single training run ($k = 1$), so its between-run variance is not estimated. With 400 test samples, differences of a few hundredths in a single metric should not be over-interpreted.

Learning-rate grid. The largest learning rate tested, 2×10^{-4}, is the best for all three models, so the optimum may lie outside the grid; for Llama the gain over 10^{-4} is already small (0.786 versus 0.777), which suggests that its curve is flattening. Only the released and the paper's learning rates were trained with three seeds; 10^{-5} and 2×10^{-4} are single runs. Because the released split has no validation set, the grid is evaluated on the test split and is not used for model selection.

Scope of cost measurements. Costs are measured for the reasoner only, with the bitsandbytes runtime on a single GPU. The perception stage (vision-language model and speech recognition), deployment runtimes and on-device measurements are out of scope.

Pruning. Pruning was evaluated on Llama-3.1-8B-Instruct only, structured pruning was restricted to MLP dimensions because preliminary experiments with attention-head pruning degraded the model severely, and recovery used LoRA rather than full fine-tuning. The original recovery adapters were trained without a sparsity constraint and the mask was re-applied only after merging; because the unconstrained update is spread evenly over pruned and kept weights, re-applying the mask discards a share of it equal to the sparsity, for all ten pruning settings and all module types (49.7% of its squared norm at 50% Wanda sparsity; Table 21). Mask-aware recovery, which restricts the update to the unpruned weights, recovers most of the resulting loss, and with the paper's learning rate and $r = 32$ it shows no significant difference to the dense model at 50% and 4:8 sparsity (Table 12 and **??**); these are single runs, the 2:4 run still produces no parseable score for 4% of the samples, and the masked update roughly triples the recovery training time. The error cascade in Section E.1 shows where the remaining losses arise: magnitude pruning and Wanda 2:4 break the output format, which mask-aware recovery restores; structured activation pruning corrupts the proactive decision; and moderate unstructured pruning, like mask-aware recovery, mainly degrades tool selection. Finally, all pruned models store their weights in dense format, saving neither memory nor latency in our setup; PyTorch's 2:4 sparse kernels reduced the allocated weight memory but made decoding about 2.8 times slower, also for the mask-aware models (Appendix C).

Knowledge distillation. Our response-based distillation from each 7–8B model into a 1–1.5B student of the same family (cross-entropy and Kullback–Leibler objective, temperature 3, $\alpha = 0.5$, 4-bit NF4) had to truncate sequences to 1024 tokens to fit teacher and student into 24 GB, and none of the students learned the required output format. Fine-tuning 1–1.5B models of the same families directly at the full sequence length succeeds (Section 7.2), so the failure was not caused by the size of the students; truncation remains the most likely cause, although the direct fine-tuning also differs in using no teacher and no quantised student. We did not test distillation from cached teacher outputs at full length, which would show whether a teacher adds anything beyond direct fine-tuning.

7.4 Future Work

Proactive agents spend most of their time deciding *not* to act. A natural efficiency direction specific to this setting is to emit the proactive score before the thought trace and stop generation when it falls below the threshold, or to gate the full reasoner with a small model trained only on the proactive score. Our outputs allow a first estimate of the saving from score-first generation. The fine-tuned models predict that no

assistance is needed for 174–195 of the 400 test samples, and on these samples they generate 86–93 tokens in total, most of them for the thought trace that precedes the score. Emitting only the score block (about 22 tokens) on these samples would reduce the number of generated tokens by 20–26%, and, with the measured per-token latency (61–66 ms per token, latency linear in output length with $r = 1.00$), the average time per sample from 9.07 to 6.93 s for Llama, from 8.77 to 7.06 s for DeepSeek and from 8.36 to 6.40 s for Qwen at the released configuration (19–25% less). Task metrics would not change, because a negative prediction carries no tool calls; the estimate assumes that a model trained to emit the score first predicts the same scores. For gating, the 1–1.5B models of Section 7.2 are a natural starting point, since they already reach high proactive accuracy (Acc-P up to 0.925) at a quarter of the memory. Mask-aware recovery combined with sparse kernels that are efficient at batch size one, or with batched monitoring, could turn the preserved accuracy of pruned models into actual memory and latency savings. Measuring cost per hour of monitoring rather than per sample, and evaluating post-training quantisation kernels and on-device runtimes, would make the efficiency conclusions directly applicable to deployment.

8 Conclusion

We reproduced *ContextAgent* on *ContextAgentBench* with the authors' released artefacts. The central claims hold: supervised fine-tuning with personas and thought traces clearly outperforms in-context learning, and the in-context baselines reproduce closely. The fine-tuned models, however, reach substantially higher scores than reported, and the size of the gap depends on a learning-rate discrepancy between the paper and the released code. Training with the effective batch size of eight-GPU data-parallel training reproduces the reported Llama score, but not those of DeepSeek and Qwen. Users of the benchmark should therefore treat the reported fine-tuned scores as a lower bound and report the learning rate, effective batch size and number of optimiser steps.

Under resource constraints, 4-bit quantisation of the base model combined with an adequate LoRA configuration shows no significant loss in tool-calling F1 for any of the three models at half the inference memory, at the cost of higher latency with on-the-fly dequantisation. Unstructured pruning preserves performance up to 30% sparsity, provided the sparsity survives the recovery step; restricting the recovery update to the unpruned weights reduces the loss at 50% and 2:4 sparsity from 0.10–0.27 to 0.03–0.09 F1, and with the paper's learning rate and $r = 32$ to no significant loss at 50% and 4:8 sparsity, but without sparse kernels pruning brings no efficiency gain, and structured pruning yields efficiency gains only at a large and sometimes silent loss of proactive behaviour. A 1.5B model fine-tuned at the full sequence length, in contrast, matches the released 7B configuration at a quarter of the memory (three seeds), and keeps this level with a 4-bit base at 2.2 GB, so for this task a smaller base model is the most effective way to reduce cost. Efficient proactive agents therefore need methods that reduce cost while preserving the decision to act and the structured output required to do so.

Code availability

Code, configuration files and raw evaluation outputs are available at an anonymised repository for review: `https://anonymous.4open.science/r/tmlr-sub-eval/`.

References

Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning—taxonomy, comparison, analysis, and recommendations, 2024. URL `https://arxiv.org/abs/2308.06767`.

Pierre V. Dantas, Lucas C. Cordeiro, and Waldir S. S. Junior. A review of state-of-the-art techniques for large language model compression, 2025. URL `https://doi.org/10.1007/s40747-025-02019-z`.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL `https://arxiv.org/abs/2208.07339`.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

Christopher Diebel, Marc Goutier, Martin Adam, and Alexander Benlian. When AI-based agents are proactive: Implications for competence and system satisfaction in human–AI collaboration, 2024. URL https://doi.org/10.1007/s12599-024-00918-y.

Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. DepGraph: Towards any structural pruning, 2023. URL https://arxiv.org/abs/2301.12900.

Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. *CoRR*, abs/1503.02531, 2015. URL http://arxiv.org/abs/1503.02531.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Gun Il Kim, Sunga Hwang, and Beakcheol Jang. Efficient compressing and tuning methods for large language models: A systematic literature review, 2025. URL https://doi.org/10.1145/3728636.

Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive agent: Shifting LLM agents from reactive responses to active assistance, 2024. URL https://arxiv.org/abs/2410.12361.

Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks, 2021. URL https://arxiv.org/abs/2104.08378.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models, 2024. URL https://arxiv.org/abs/2306.11695.

Bufang Yang, Lilin Xu, Liekang Zeng, Yunqi Guo, Siyang Jiang, Wenrui Lu, Kaiwei Liu, Hancheng Xiang, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. ProAgent: Harnessing on-demand sensory contexts for proactive LLM agent systems, 2025a. URL https://arxiv.org/abs/2512.06721v1.

Bufang Yang, Lilin Xu, Liekang Zeng, Kaiwei Liu, Siyang Jiang, Wenrui Lu, Hongkai Chen, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. ContextAgent: Context-aware proactive LLM agents with open-world sensory perceptions, 2025b. URL https://arxiv.org/abs/2505.14668v2.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

A Acronyms

AWQ Activation-aware Weight Quantisation

Acc-Args Argument Accuracy

Acc-P Proactive Prediction Accuracy

FD False Detection rate

GPTQ post-training quantisation for Generative Pre-trained Transformers

KD Knowledge Distillation

LLM Large Language Model

LoRA Low-Rank Adaptation

MD Missed Detection rate

MLP Multilayer Perceptron

NF4 4-bit NormalFloat

QLoRA Quantised Low-Rank Adaptation

RMSE Root Mean Square Error

SFT Supervised Fine-Tuning

TOST two one-sided tests

WANDA Pruning by **W**eights **and** **A**ctivations

B Training Costs

Table 14: Training cost of the reproduced *ContextAgent* models (released configuration). Runs with the exact released configuration (no LoRA dropout, no gradient checkpointing).

		Costs		
Method	Model	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
ContextAgent	Llama-3.1-8B-Ins	2691.0	21.02	17.79
	DeepSeek-R1-7B	3446.6	21.02	16.68
	Qwen2.5-7B-Ins	2862.2	21.35	17.08

Table 15: Training cost under base-model quantisation (LoRA $r = 8$, learning rate 5×10^{-5}). All rows, including 16-bit, use the settings of the LoRA experiments (LoRA dropout 0.05, gradient checkpointing; Section 5.3), so durations and memory are not directly comparable to Table 14.

		Costs		
Model	Q-Bit	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
LLaMA-3.1-8B-Ins	16-bit	**2855.8**	21.10	18.94
	8-bit	3656.8	19.07	18.75
	4-bit	3248.1	**11.28**	**10.77**
DeepSeek-R1-7B	16-bit	**2687.8**	21.12	18.12
	8-bit	3398.6	19.95	**13.6**
	4-bit	3474.7	**18.91**	16.15
Qwen2.5-7B-Ins	16-bit	**2935.6**	21.37	17.31
	8-bit	3609.9	19.95	**13.9**
	4-bit	3211.3	**15.95**	15.18

Table 16: Training cost of the combined configurations.

Model	Config	Costs		
		Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
LLaMA-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	3006.5	21.63	18.67
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	3305.7	12.24	11.73
DeepSeek-R1-7B	$r = 64, lr = 1.0 \times 10^{-4}$, all, 16-bit	2725.0	21.22	20.57
	$r = 64, lr = 1.0 \times 10^{-4}$, all, 4-bit	3402.6	11.54	11.16
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	2985.1	21.91	18.23
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	2996.5	15.26	14.69

Table 17: Recovery training cost after pruning (`Llama-3.1-8B-Instruct`). [‡]Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

Structure	Method	%	Costs		
			Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
Unstructured	Wanda	10	2787.0	19.54	18.95
		30	2770.5	19.54	18.90
		50	2765.2	19.54	18.87
		50[‡]	8489.4	19.76	19.49
		50[‡], 10^{-4}, r=8	8379.0	19.76	19.46
		50[‡], 10^{-4}, r=32	8338.9	20.36	20.04
	Magnitude	10	2782.0	19.54	18.79
		30	2751.5	19.54	18.93
		50	2710.2	19.54	18.97
		50[‡]	8498.9	19.76	19.49
Structured	Activation	10	2891.5	19.71	18.84
		30	1986.3	15.33	14.55
		50	1479.3	11.14	10.64
	Magnitude	10	2745.0	19.71	18.83
		30	1926.3	15.33	14.43
		50	1524.0	11.14	10.36

C Semi-structured Pruning

Semi-structured pruning aims to balance the flexibility of unstructured pruning with the hardware efficiency of structured sparsity. A widely adopted formulation is **N:M** sparsity, in which exactly N non-zero weights are retained within each contiguous block of M parameters (Mishra et al., 2021). This specific pattern is highly relevant for modern hardware accelerators, such as GPUs equipped with Sparse Tensor Cores, which natively accelerate 2:4 sparsity to achieve tangible inference speedups.

Table 18: Semi-structured pruning (50% sparsity) of `Llama-3.1-8B-Instruct` after recovery fine-tuning. The pruning mask is re-applied after merging the recovery adapter; re-masked Wanda rows show the mean of three evaluations. ‡Mask-aware recovery: the LoRA update is restricted to the unpruned weights (Section 5.5; single runs). Rows marked 10^{-4}: mask-aware recovery with learning rate 10^{-4} and the stated rank instead of the released recovery configuration ($r = 8$, 5×10^{-5}).

			Proactive Predictions				Tool Calling			
Structure	Pruning	%	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Semi-structured	Wanda	2:4 (50%)	0.703	0.162	0.135	2.291	0.511	0.458	0.469	0.185
		4:8 (50%)	0.862	0.094	0.044	1.423	0.669	0.608	0.620	0.432
		2:4 (50%)‡	0.875	0.045	0.080	1.343	0.664	0.665	0.649	0.455
		4:8 (50%)‡	0.868	0.083	0.050	1.323	0.688	0.673	0.666	0.490
		2:4 (50%)‡, 10^{-4}, r=8	0.900	0.043	0.058	1.216	0.725	0.730	0.711	0.552
		4:8 (50%)‡, 10^{-4}, r=8	0.927	0.043	**0.030**	1.034	0.763	0.756	0.745	0.587
		2:4 (50%)‡, 10^{-4}, r=32	0.915	0.048	0.037	1.124	0.758	0.758	0.744	0.591
		4:8 (50%)‡, 10^{-4}, r=32	**0.940**	**0.030**	**0.030**	**0.929**	**0.783**	**0.777**	**0.761**	**0.613**
	Magnitude	2:4 (50%)	0.805	0.080	0.115	1.740	0.530	0.518	0.517	0.276
		4:8 (50%)	0.768	0.163	0.070	1.810	0.571	0.538	0.544	0.305
		2:4 (50%)‡	0.875	0.072	0.052	1.388	0.680	0.676	0.666	0.473
		4:8 (50%)‡	0.882	0.062	0.055	1.339	0.711	0.707	0.693	0.522

Table 18 shows semi-structured pruning at 50% sparsity. With the original recovery and the pruning mask re-applied afterwards, all four settings lose substantially against the dense model (seed-42 run, F1-score 0.735; all differences significant). The less constrained 4:8 pattern is better than 2:4 for WANDA (F1-score 0.620 versus 0.469, Acc-Args 0.432 versus 0.185) and slightly better for magnitude pruning (0.544 versus 0.517); with 2:4, both criteria fail similarly, with Acc-P falling to 0.70–0.81. Mask-aware recovery (rows marked ‡) improves all four settings significantly, by +0.046 (WANDA 4:8) to +0.180 (WANDA 2:4) in F1-score, and removes the collapse of the 2:4 pattern: all four settings then reach an F1-score of 0.649–0.693 and an Acc-P of at least 0.868. The differences between the two criteria and the two patterns shrink to at most 0.044 in F1-score, and magnitude 4:8 performs best (0.693). All four remain significantly below the dense model (-0.042 to -0.086) and slightly below unstructured WANDA at the same sparsity with mask-aware recovery (0.706).

With the paper's learning rate of 10^{-4}, mask-aware recovery of the WANDA patterns improves further. At $r = 8$ the F1-score rises to 0.711 (2:4) and 0.745 (4:8), significantly above mask-aware recovery at 5×10^{-5} (+0.062, 95% CI $[0.024, 0.100]$, and +0.079, $[0.045, 0.114]$); at $r = 32$ it reaches 0.744 and 0.761. Against the dense model trained at the same learning rate (seed 42, $r = 8$, F1-score 0.781), 4:8 with $r = 32$ shows no significant difference (-0.020, $[-0.051, 0.011]$), while 2:4 remains slightly below (-0.037, $[-0.070, -0.005]$); the 2:4 run is also the only mask-aware run with a noticeable share of outputs without a parseable proactive score (4.0%, Section E.1). These are single runs.

Table 19: Recovery training cost after semi-structured pruning. ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

Structure	Method	%	Costs		
			Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
Semi-structured	Wanda	2:4 (50%)	2719.8	**19.54**	18.82
		4:8 (50%)	2732.5	**19.54**	18.91
		2:4 (50%)‡	8437.5	19.76	19.50
		4:8 (50%)‡	8468.4	19.76	19.48
		2:4 (50%)‡, 10^{-4}, r=8	8430.2	19.76	19.48
		4:8 (50%)‡, 10^{-4}, r=8	8298.0	19.76	19.48
		2:4 (50%)‡, 10^{-4}, r=32	8442.4	20.36	20.07
		4:8 (50%)‡, 10^{-4}, r=32	8320.7	20.36	20.14
	Magnitude	2:4 (50%)	**2700.6**	**19.54**	18.96
		4:8 (50%)	2726.3	**19.54**	18.89
		2:4 (50%)‡	8478.8	19.76	19.46
		4:8 (50%)‡	8479.0	19.76	19.48

Training costs are reported in Table 19. With the original recovery, peak GPU memory is 19.54 GB for all configurations, as for unstructured pruning, and training times range from 2701 to 2733 seconds. Mask-aware recovery needs about 3.1 times as long (8298–8479 s) at a slightly higher peak of 19.76 GB (20.36 GB with $r = 32$), because the masked update is formed as a full matrix in every forward pass; the learning rate does not change this cost.

Table 20: Inference cost after semi-structured pruning. "Dense storage": the pruned weights are stored and multiplied densely. "Sparse kernels": the 2:4 layers are executed with PyTorch's semi-structured sparse tensor-core kernels (`to_sparse_semi_structured`). ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

Structure	Method	%	Costs					
			GPU Peak (GB)↓	GPU Avg (GB)↓	Avg Batch (s)↓	Min Batch (s)↓	Max Batch (s)↓	Avg Samples/s↑
Semi-structured (dense storage)	Wanda	2:4 (50%)	15.7	15.7	8.7	1.9	69.0	0.1150
		4:8 (50%)	15.7	15.7	7.6	3.0	68.4	0.1324
		2:4 (50%)‡	15.6	15.6	9.6	4.0	37.8	0.1040
		4:8 (50%)‡	15.6	15.6	10.3	4.4	27.5	0.0975
		2:4 (50%)‡, 10^{-4}, r=8	15.6	15.6	9.7	4.1	28.3	0.1027
		4:8 (50%)‡, 10^{-4}, r=8	15.6	15.6	9.8	3.1	26.3	0.1019
		2:4 (50%)‡, 10^{-4}, r=32	15.6	15.6	10.3	3.0	23.9	0.0971
		4:8 (50%)‡, 10^{-4}, r=32	15.6	15.6	9.9	4.0	23.1	0.1009
	Magnitude	2:4 (50%)	15.7	15.7	9.0	3.6	69.0	0.1116
		4:8 (50%)	15.7	15.7	8.2	2.3	68.6	0.1223
		2:4 (50%)‡	15.7	15.6	9.4	4.1	68.7	0.1061
		4:8 (50%)‡	15.6	15.6	9.3	4.0	28.5	0.1078
Sparse kernels	Wanda	2:4 (50%)	15.8	15.8	24.4	3.8	192.0	0.0410
		2:4 (50%)‡	15.8	15.8	24.9	9.6	57.9	0.0402
	Magnitude	2:4 (50%)	15.8	15.8	24.6	8.7	192.7	0.0407
		2:4 (50%)‡	15.8	15.8	25.3	10.1	54.7	0.0395

At inference (Table 20), peak GPU memory remains 15.6–15.7 GB and the average batch time 7.6–10.3 s, similar to the dense model (9.0 s), also with mask-aware recovery; differences come from output length, not from the kernels. To test whether hardware support changes this, we executed the 2:4 models with PyTorch's semi-structured sparse kernels, which use the sparse tensor cores of the L4 GPU: all 224 linear layers of each model are converted after loading, and the task metrics are unchanged within evaluation noise (re-masked: F1-score 0.487 and 0.503; mask-aware: 0.645 and 0.682, not significantly different from the same models with dense execution, -0.004, 95% CI $[-0.042, 0.033]$, and $+0.016$, $[-0.019, 0.052]$). Generation, however, becomes about 2.8 times slower (24.4–25.3 s per sample instead of 8.7–9.6 s), most likely because with a batch size of one every decoding step multiplies a single token vector, a shape for which the sparse kernels are less efficient than dense matrix multiplication. The compressed weights reduce the allocated GPU memory from 15.0

to 9.3 GiB (−38%), but because the dense weights are loaded first and converted afterwards, the memory reserved by the process, and thus the measured peak, stays at 15.8 GB. Realising both savings would require loading the weights directly in compressed form and batched inference.

How much of the recovery update re-masking discards. For each unconstrained recovery adapter and its pruned base model, with mask M (1 = kept) and update $D = BA$, we measure the share of the update that re-applying the mask removes, $\|(1 - M) \odot D\|_F^2 / \|D\|_F^2$, summed over all 224 adapted layers (Table 21). For every adapter it equals the sparsity, and it is the same for all module types: the unconstrained update is spread evenly over pruned and kept weights, so re-masking discards a share equal to the sparsity. Mask-aware recovery avoids this loss by construction, which is why it helps most at 50% and with the 2:4 pattern.

Table 21: Share of the unconstrained recovery update removed by re-applying the pruning mask ($\|(1 - M) \odot BA\|_F^2 / \|BA\|_F^2$ over all adapted layers), and its range over the seven module types. "Measured sparsity": share of pruned weights in the adapted layers (magnitude pruning thresholds globally).

Criterion	Pattern	Measured sparsity	Discarded share	Range over modules
Wanda	10%	0.100	0.099	0.099–0.100
	30%	0.300	0.298	0.297–0.299
	50%	0.500	0.497	0.496–0.499
	2:4	0.500	0.498	0.497–0.499
	4:8	0.500	0.497	0.496–0.499
Magnitude	10%	0.100	0.100	0.100–0.102
	30%	0.301	0.301	0.300–0.306
	50%	0.502	0.502	0.501–0.512
	2:4	0.500	0.500	0.500–0.501
	4:8	0.500	0.500	0.500–0.501

D Statistical Analysis

Figure 3 shows the paired-bootstrap differences in tool-calling F1-score, with 95% confidence intervals, for all comparisons discussed in the paper (method in Section 5.6).

The learning-rate effect is significant for all three models (+0.037, +0.089 and +0.033, averaged over three seeds). Among the pruning settings with re-masked recovery, Wanda at 50% sparsity, all semi-structured settings and magnitude pruning at 30% and 50% lie clearly below the dense model; mask-aware recovery improves all six settings at 50% sparsity significantly, and Wanda at 50% is then no longer significantly below the dense model. With the paper's learning rate, none of the six mask-aware runs (Wanda at 50%, 2:4 and 4:8, each with $r = 8$ and $r = 32$) is significantly below the dense model at 5×10^{-5}, and Wanda at 50% with $r = 32$ is significantly above it (+0.036); against the dense model at 10^{-4}, the $r = 32$ runs at 50% and 4:8 show no significant difference, whereas all three $r = 8$ runs and 2:4 with $r = 32$ remain significantly below. All quantisation comparisons, at the released default and at the combined configuration (three seeds each) and for Qwen2.5-1.5B (4-bit versus three 16-bit seeds), the higher rank of the combined configurations compared with $r = 8$ at 10^{-4} (three seeds each), as well as Wanda at 10% and 30% and magnitude pruning at 10%, have confidence intervals that include zero: these differences are not significant. Equivalence within ±0.03 is shown for 8-bit Llama, for the 4-bit combined configuration of DeepSeek and for the exact released configuration of DeepSeek and Qwen; the other intervals extend beyond the margin, so we report these results as no significant difference rather than as equivalence.

To test whether the ranking of methods reproduces, we compare our F1-scores in Table 1 with those reported by Yang et al. (2025b) for the five methods and three models. Kendall's τ is 0.80 for each model ($p = 0.08$, with only five methods per model) and 0.82 over all 15 method–model pairs ($p < 0.001$).

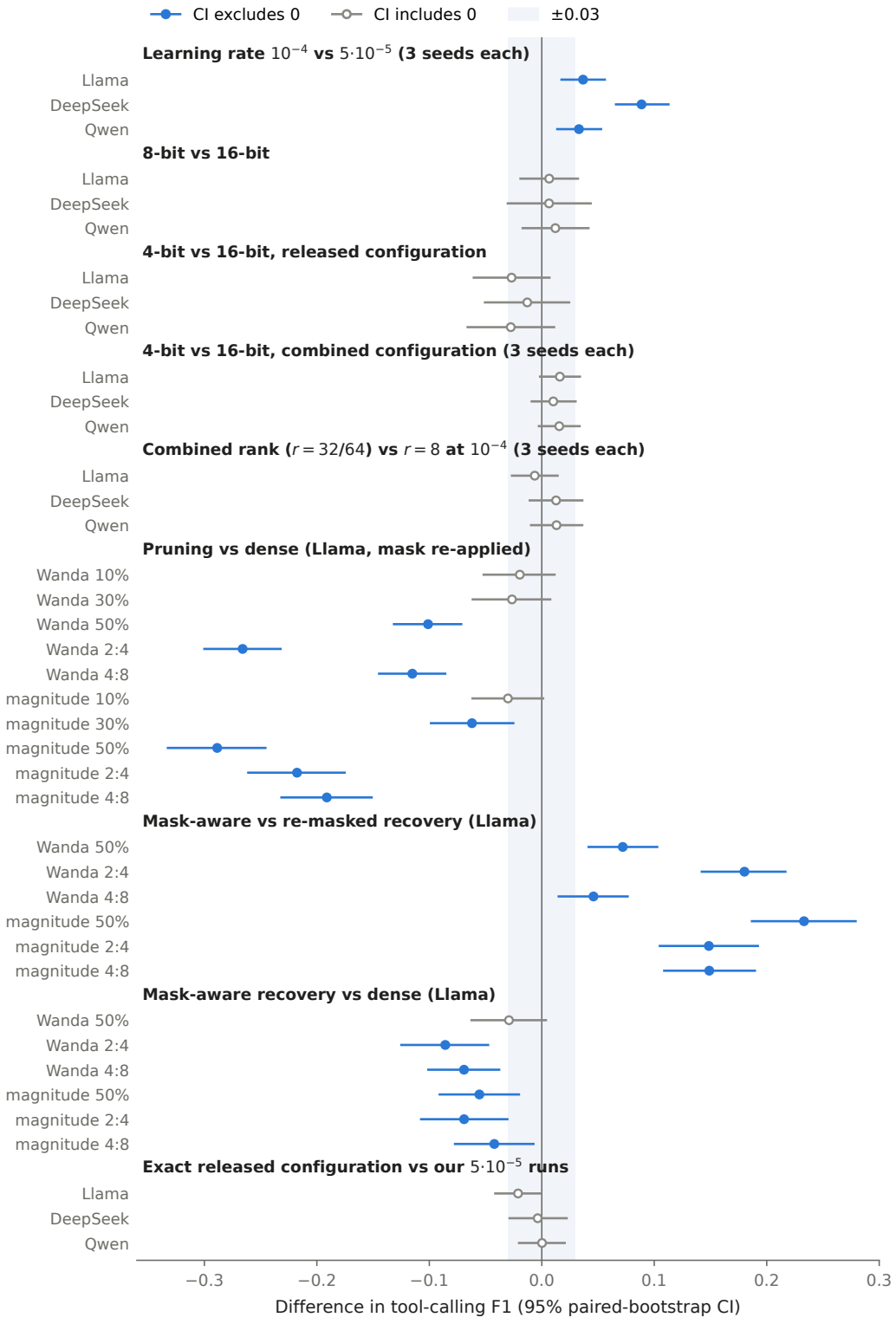


Figure 3: Paired-bootstrap differences in tool-calling F1-score with 95% confidence intervals over the 400 test samples. Filled blue: the interval excludes zero; hollow grey: it includes zero (no significant difference). Shaded band: ±0.03. Learning-rate, combined-configuration and rank comparisons average three seeds per configuration; pruning comparisons are against the seed-42 dense model; mask-aware recovery comparisons are single runs. The last groups compare mask-aware recovery at learning rate 10^{-4} ($r = 8$ and $r = 32$; single runs) with the dense model at 5×10^{-5} and at 10^{-4} (seed 42), and 4-bit with 16-bit Qwen2.5-1.5B.

E Where the Models Fail

E.1 Error cascade

To show *how* the configurations fail, we assign every test sample to exactly one stage of the decision pipeline (Table 22): (a) no parseable proactive score; (b) a parseable but wrong proactive decision (missed or false detection); (c) a correct decision that no assistance is needed; (d) a correct proactive trigger with a wrong or incomplete tool set; (e) a correct trigger and the exact tool set, but at least one wrong tool argument (the rule of Acc-Args: the tool is not in the ground-truth chain or its sandbox call returns an error); and (f) a fully correct response. With several runs per configuration, the shares are averaged over runs.

Table 22: Error cascade on the 400 test samples (% of samples; each sample is counted in exactly one stage). (a) no proactive score; (b) wrong proactive decision; (c) correct non-proactive decision; (d) correct trigger, wrong tool set; (e) correct trigger and tool set, wrong arguments; (f) fully correct. LoRA rows: $r = 8$, mean over three seeds; pruning rows: `Llama-3.1-8B-Instruct` with the pruning mask re-applied, mean over the repeated evaluations where available; ‡mask-aware recovery, rows marked 10^{-4} with learning rate 10^{-4} and the stated rank; 1–1.5B rows with 16-bit base: mean over three seeds.

Model	Configuration	Runs	(a)	(b)	(c)	(d)	(e)	(f)
Llama-3.1-8B-Ins	ICL-All	1	12.8	20.2	35.2	28.2	0.0	3.5
	LoRA 5×10^{-5} (code)	3	0.0	8.5	42.9	34.2	0.0	14.4
	LoRA 10^{-4} (paper)	3	0.0	7.2	43.8	32.0	0.0	17.0
	4-bit, 5×10^{-5}	1	0.0	10.8	40.0	36.8	0.0	12.5
DeepSeek-R1-7B	ICL-All	1	55.5	8.8	6.5	27.8	0.5	1.0
	LoRA 5×10^{-5} (code)	3	0.0	14.5	38.1	39.9	0.0	7.5
	LoRA 10^{-4} (paper)	3	0.1	11.1	40.7	35.1	0.1	13.0
	4-bit, 5×10^{-5}	1	0.0	15.8	37.2	39.2	0.0	7.8
Qwen2.5-7B-Ins	ICL-All	1	40.8	6.5	9.2	38.5	0.2	4.8
	LoRA 5×10^{-5} (code)	3	0.1	8.8	42.2	37.4	0.0	11.5
	LoRA 10^{-4} (paper)	3	0.0	7.5	43.9	34.2	0.0	14.3
	4-bit, 5×10^{-5}	1	20.0	8.2	27.8	32.8	0.0	11.2
Llama-3.1-8B-Ins (pruned)	Wanda 10% / 30% / 50%	1/1/3	0.0 / 0.0 / 0.2	10.2 / 8.0 / 11.2	40.0 / 43.8 / 42.2	34.0 / 37.5 / 38.2	0.0 / 0.0 / 0.0	15.8 / 10.8 / 8.1
	Magnitude 10% / 30% / 50%	1	0.0 / 0.0 / 92.5	8.5 / 9.2 / 1.5	42.8 / 40.8 / 1.2	35.8 / 38.2 / 4.8	0.0 / 0.0 / 0.0	13.0 / 11.8 / 0.0
	Wanda 2:4 / 4:8	3	34.0 / 0.9	15.7 / 13.2	13.5 / 42.2	33.1 / 36.0	0.9 / 0.3	2.8 / 7.2
	Magnitude 2:4 / 4:8	1	3.0 / 6.2	17.0 / 18.8	35.0 / 38.2	41.5 / 34.0	1.0 / 0.2	2.5 / 2.5
	Wanda 50% / 2:4 / 4:8‡	1	0.0 / 0.0 / 0.5	10.5 / 12.5 / 12.8	43.8 / 39.0 / 42.0	34.5 / 39.8 / 34.8	0.0 / 0.2 / 0.0	11.2 / 8.5 / 10.0
	Wanda 50% / 2:4 / 4:8‡, 10^{-4}, r=8	1	0.2 / 0.2 / 0.8	8.8 / 9.8 / 7.2	43.8 / 41.2 / 43.2	34.5 / 36.0 / 33.5	0.0 / 0.0 / 0.0	12.8 / 12.8 / 15.2
	Wanda 50% / 2:4 / 4:8‡, 10^{-4}, r=32	1	0.0 / 4.0 / 0.0	7.0 / 7.8 / 6.0	43.5 / 40.0 / 44.0	32.0 / 32.8 / 36.2	0.0 / 0.0 / 0.0	17.5 / 15.5 / 13.8
	Magnitude 50% / 2:4 / 4:8‡	1	0.5 / 0.2 / 0.0	11.0 / 12.2 / 11.8	40.5 / 41.8 / 41.5	37.0 / 35.0 / 37.0	0.0 / 0.0 / 0.0	11.0 / 10.8 / 9.8
	Structured activation 10% / 30% / 50%	1	0.0 / 0.5 / 0.5	13.0 / 28.2 / 47.5	42.2 / 35.5 / 30.5	38.0 / 33.5 / 21.5	0.0 / 0.0 / 0.0	6.8 / 2.2 / 0.0
	Structured magnitude 10% / 30% / 50%	1	100 / 100 / 100	0	0	0	0	0
1–1.5B, 2048 tokens	Llama-3.2-1B, $r = 32$	3	0.1	12.5	41.8	35.2	0.0	10.3
	DeepSeek-R1-1.5B, $r = 32$	3	0.3	13.8	39.5	39.2	0.2	6.9
	Qwen2.5-1.5B, $r = 32$	3	0.0	9.7	43.1	35.9	0.0	11.3
	Qwen2.5-1.5B, $r = 32$, 4-bit	1	0.0	9.5	42.0	34.5	0.0	14.0

Three findings stand out. First, once a model is fine-tuned, practically all of its errors are tool-selection errors: stage (e) stays at or below 0.1% for every fine-tuned 16-bit 7–8B model (and at or below 0.2% for the 1–1.5B models), so a correct tool set almost always comes with correct arguments, and the differences in Acc-Args between configurations stem from choosing the wrong tools, not from filling them in wrongly. The paper's learning rate improves all three models in the same way, moving samples from (b) and (d) to (f) (fully correct: Llama 14.4% to 17.0%, DeepSeek 7.5% to 13.0%, Qwen 11.5% to 14.3%). Second, the in-context baselines fail mainly on the output format: 55.5% (DeepSeek) and 40.8% (Qwen) of their outputs contain no parseable proactive score, which fine-tuning removes completely. Third, the compressed models fail in different ways. With the original, re-masked recovery, 50% magnitude pruning and Wanda 2:4 cause a *format collapse* (no parseable score for 92.5% and 34% of the samples); mask-aware recovery removes it (0.5% and 0%), so the collapse is caused by the recovery procedure rather than by the sparsity. With the paper's learning rate and $r = 32$, mask-aware recovery moves further samples from (b) to (f) (fully correct: 17.5% for Wanda at 50%, as for the dense model at 10^{-4} with 17.0%); the only exception is 2:4, where 4.0% of the outputs again lack a parseable score. Structured magnitude pruning loses the format completely (100%) at every ratio, structured activation pruning mainly causes *wrong proactive decisions* (stage (b) rising from 13.0% to 28.2% and 47.5%), and moderate unstructured pruning, mask-aware recovery and the 1–1.5B models keep the decisions intact and lose mainly on *tool selection*. Quantisation mostly behaves like the 16-bit model, with one exception:

4-bit Qwen at the released learning rate produces no parseable score on 20% of the samples, which explains its significantly lower Acc-P (Section 6.2.2); the 4-bit Qwen2.5-1.5B with $r = 32$ shows no such failure (0.0%).

Table 23: Tool-calling F1-score and Acc-Args by the length of the ground-truth tool chain. For samples that need no tool (187 of 400), F1 is the share of correct "no tool" predictions and Acc-Args is undefined. LoRA rows: $r = 8$, mean over three seeds; ‡mask-aware recovery.

		0 tools ($n = 187$)	1 tool ($n = 72$)		2 tools ($n = 65$)		≥3 tools ($n = 76$)	
Model	Configuration	F1	F1	Acc-Args	F1	Acc-Args	F1	Acc-Args
Llama-3.1-8B-Ins	ICL-All	0.930	0.227	0.231	0.278	0.357	0.197	0.341
	LoRA 5×10^{-5}	0.913	0.588	0.454	0.587	0.643	0.591	0.738
	LoRA 10^{-4}	0.930	0.651	0.530	0.623	0.691	0.650	0.806
DeepSeek-R1-7B	ICL-All	0.909	0.097	0.088	0.183	0.209	0.117	0.273
	LoRA 5×10^{-5}	0.818	0.374	0.274	0.422	0.428	0.409	0.523
	LoRA 10^{-4}	0.870	0.485	0.355	0.546	0.563	0.537	0.698
Qwen2.5-7B-Ins	ICL-All	0.914	0.259	0.273	0.387	0.484	0.244	0.417
	LoRA 5×10^{-5}	0.893	0.492	0.365	0.558	0.597	0.581	0.740
	LoRA 10^{-4}	0.930	0.527	0.401	0.584	0.641	0.607	0.767
Llama (pruned)	Wanda 30%	0.925	0.498	0.443	0.524	0.589	0.533	0.780
	Wanda 50%	0.888	0.466	0.380	0.357	0.420	0.405	0.681
	Wanda 50%‡	0.930	0.493	0.395	0.537	0.566	0.498	0.686
	Structured activation 10%	0.893	0.368	0.272	0.436	0.450	0.434	0.577
	Structured activation 30%	0.765	0.150	0.114	0.248	0.285	0.298	0.438
Qwen2.5-1.5B-Ins	LoRA $r = 32$, 10^{-4}	0.936	0.440	0.345	0.544	0.570	0.536	0.743

Table 23 separates the samples by the number of tools the ground truth requires. Almost all configurations recognise most samples that need no tool (F1 0.82–0.94; only structured activation pruning at 30% drops to 0.77), so the differences between configurations come mainly from the 213 samples that require tools. There, the in-context baselines reach an F1-score of only 0.10–0.39 regardless of chain length, whereas the fine-tuned models score 0.37–0.65 with little dependence on the length of the chain. The gain from the paper's learning rate is spread over all chain lengths: for Llama it is largest for single-tool and long chains (+0.06 each), and for DeepSeek, which gains most, it amounts to +0.11 to +0.13 at every length. Acc-Args rises with the chain length for every fine-tuned model (e.g. Llama at 10^{-4}: 0.53, 0.69, 0.81), because the metric is computed per tool call: a missed proactive action counts as a single wrong call, which weighs more in a sample with one ground-truth tool than in a sample with three or more. With re-masked recovery, pruning hurts the longer chains first: at 50% Wanda sparsity, F1 for two-tool chains falls from 0.524 (30%) to 0.357, while single-tool chains fall only from 0.498 to 0.466; mask-aware recovery restores the two-tool chains to 0.537. Qwen2.5-1.5B with $r = 32$ stays below Qwen2.5-7B at 10^{-4} at every chain length, by 0.04–0.09 in F1-score, most for single-tool samples (0.440 versus 0.527).

F Small Models at Full Sequence Length

Table 24: 1–1.5B models fine-tuned at the full sequence length of 2048 tokens (LoRA on all linear layers, learning rate 10^{-4}, five epochs; $r = 8$: single runs; $r = 32$: mean ± standard deviation over three seeds; 4-bit: NF4 base model, single run). Inference cost at batch size 1 on the L4 GPU, averaged over the runs; the time per sample varies between runs of the same configuration (e.g. 3.7–4.7 s for Qwen2.5-1.5B), while the memory is identical.

		Proactive Predictions				Tool Calling				Inference	
Model	Rank	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑	GPU (GB)↓	s/sample↓
Llama-3.2-1B-Ins	8	0.820	0.105	0.075	1.576	0.636	0.645	0.627	0.417	3.3	2.5
	32	0.875 ± 0.005	0.074 ± 0.008	0.051 ± 0.004	1.337 ± 0.041	0.686 ± 0.007	0.688 ± 0.005	0.673 ± 0.007	0.468 ± 0.013	3.3	3.4
DeepSeek-R1-1.5B	8	0.835	0.060	0.105	1.550	0.524	0.545	0.525	0.267	3.8	3.8
	32	0.861 ± 0.009	0.067 ± 0.006	0.073 ± 0.004	1.424 ± 0.060	0.611 ± 0.011	0.619 ± 0.006	0.603 ± 0.009	0.385 ± 0.012	3.9	4.3
Qwen2.5-1.5B-Ins	8	0.858	0.095	0.048	1.411	0.669	0.664	0.653	0.441	3.8	3.5
	32	0.903 ± 0.019	0.058 ± 0.009	**0.039 ± 0.011**	1.214 ± 0.093	0.724 ± 0.006	**0.722 ± 0.012**	0.707 ± 0.006	0.528 ± 0.024	3.8	4.2
	32, 4-bit	**0.905**	**0.045**	0.050	**1.196**	**0.730**	0.721	**0.710**	**0.547**	2.2	12.1

G Robustness Across Scenarios

All results in the main text aggregate the 400 test samples over the nine scenarios of *ContextAgentBench*. To test whether the efficiency techniques degrade performance uniformly or whether some scenarios collapse, Figure 4 breaks the tool-calling F1-score down by scenario (31–54 test samples each). The scenarios differ clearly in difficulty: for the dense Llama model, Health (0.58) and Chitchat (0.62) are the hardest and Leisure (0.85) and Work (0.86) the easiest, and this ordering is largely shared by all configurations. For each technique we compute the per-scenario F1 difference to its reference with a paired bootstrap and test for heterogeneity with a permutation test on the between-scenario variance of the per-sample differences (10,000 permutations of the scenario labels).

4-bit quantisation of the combined configuration changes the F1-score by between -0.027 (Travel) and $+0.039$ (Leisure) per scenario, with no evidence of heterogeneity ($p = 0.79$). Wanda pruning to 50% sparsity with re-masked recovery lowers the F1-score in every scenario; the loss ranges from -0.034 (Leisure) to -0.173 (Cooking) and is larger in Chitchat, Cooking, Shopping and Travel than in Leisure and Outdoors, but these differences are compatible with a uniform effect ($p = 0.34$). With mask-aware recovery, the overall loss shrinks to -0.029, again without heterogeneity ($p = 0.44$). Replacing Qwen2.5-7B by Qwen2.5-1.5B at the same configuration costs -0.047 overall, also without significant heterogeneity ($p = 0.15$). With 31–54 samples per scenario, the per-scenario confidence intervals are wide (about ± 0.04 to ± 0.14), so moderate scenario-specific effects cannot be excluded, but none of the techniques causes a scenario to collapse while others remain intact.

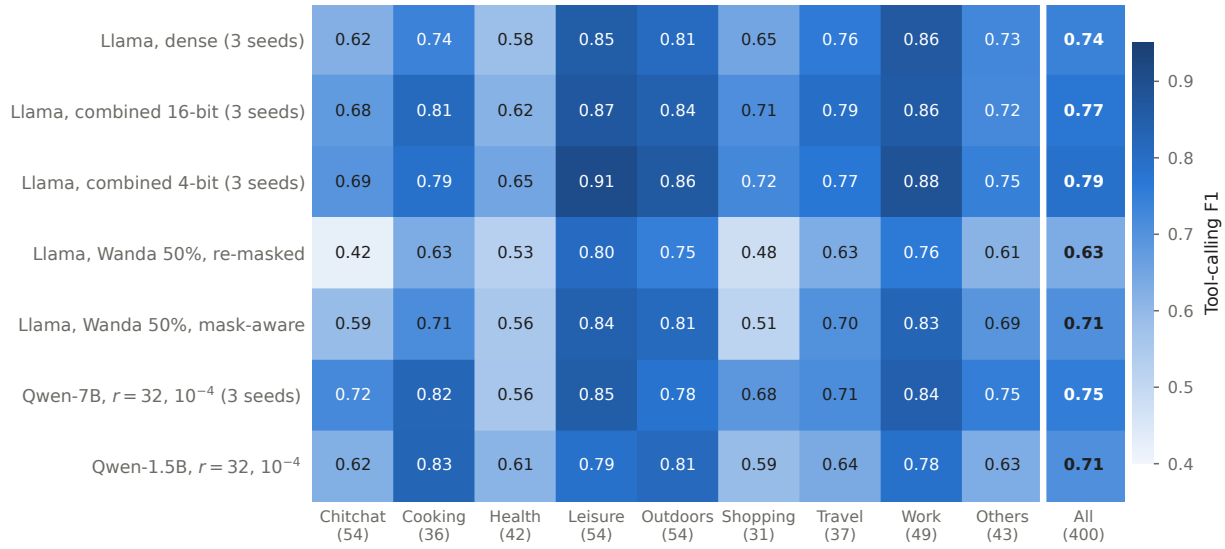


Figure 4: Tool-calling F1-score per *ContextAgentBench* scenario (number of test samples in brackets). Rows with three seeds or repeated evaluations show the mean. The last column repeats the overall score.